*A Mini Project Report on*

# AI-Based Bread Quality Indicator System

*Submitted to the*

**DEPARTMENT OF ELECTRONICS AND INSTRUMENTATION ENGINEERING**

*In partial fulfillment of the requirement of*

**Bachelor of Engineering in Electronics and Instrumentation Engineering**

*By*


| | |
|---|---|
| **SAMAY THAKUR** | **1MS23EI044** |
| **VINOD NAIK** | **1MS24EI404** |
| **NAVEEN L** | **1MS23EI034** |
| **SUBHAM KUMAR SINGH** | **1MS23EI053** |


*Under the guidance of*


**Dr. M K Pushpa, M.Tech, PhD**

**Professor, Dept. of EIE**

**Ramaiah Institute of Technology,**

*(Autonomous Institute, Affiliated to VTU)*

Vidya Soudha, MSR Nagar, MSRIT post, Bengaluru - 560054, India.


June 2026

**RAMAIAH INSTITUTE OF TECHNOLOGY**
**Department of Electronics and Instrumentation Engineering**
Bengaluru, Karnataka-560 054.

# CERTIFICATE

This is to certify that the project work entitled **"AI-Based Bread Quality Indicator System"** is carried out by **N**aveen L (IMS23E1034), Samay Thakur(IMS23E1044), Subham Kumar Singh (IMS23E1053), Vinod Naik (IMS24E1404), are bonafide students of Ramaiah Institute of Technology, Bengaluru in partial fulfillment of the requirement of the course EIP67-Mini project of **Bachelor of Engineering** in **Electronics & Instrumentation Engineering** of the Visvesvaraya Technological University, Belagavi, during the year 2024-2025. It is certified that all the corrections/suggestions indicated during Internal Assessment heve been incorporated in the report deposited in the department library. The project report has been approved as it satisfies the academic requirements with respect to Mini-Project work.

**Guide Signature**
Dr M.K Pushpa M.Tech.,Ph.D.,
Professor,
Dept. of EIE,R.I.T,
Bengaluru-560054.

**HOD Signature**
Dr Shivaprakash G M.Tech.,Ph.D.,
Professor and HOD,
Dept. of EIE,R.I.T,
Bengaluru-560054.

**External Examiners**

Name of the examiner — Signature with date

1.

2.

RAMAIAH
Institute of Technology

# Declaration

We hereby declare that the dissertation **AI-Based Bread Quality Indicator System** submitted by us to the department of Electronics and Instrumentation Engineering, RIT Bengaluru-560 054 in partial fulfillment of the requirement of the course EIP67-Mini project of **Bachelor of Engineering** in **Electronics & Instrumentation Engineering** is a bona-fide record of the work carried out by us under the supervision of ***Dr M.K Pushpa***.

We further declare that the work reported in this Mini-project report, has not been submitted and will not be submitted, either in part or in full, to this institute or of any other institute or University.

Sign: _____

Samay Thakur - 1MS23EI044

Date: _____

Sign: _____

Vinod Naik - 1MS24EI404

Date: _____

Sign: _____

Naveen L - 1MS23EI034

Date: _____

Subham Kumar Singh - 1MS23EI053

Date: _____

# *Acknowledgements*

The gratification of happiness that accompanies any successful work would be incomplete without the mention of the people who made it possible. We express our gratitude to the Guide **Dr. M.K Pushpa**, Professor,Department of Electronics and Instrumentation Engineering, for providing active support and guidance at every stage of our project. We take this opportunity to thank her for the constant support and guidance provided throughout the process of implementing the idea of " AI-Based Bread Quality Indicator System , and valuable suggestions.

We also thank our coordinator **Dr. K M Vanitha**, for letting us proceed with our mini-project. We consider it a privilege to thank **Dr.G.Shivaprakash**, Professor and Head of the department of Electronics and Instrumentation Engineering, Ramaiah Institute of Technology, Bengaluru for providing all the facilities. We thank him for sharing the LaTeX template.

We thank **Dr. N. V. R. Naidu**, Principal and Management of Ramaiah Institute of Technology, Bengaluru for all the support to carry out the project. We whole heartily thank the Management of Ramaiah Institute of Technology for providing us a vibrant learning ecosystem for our all round development. We are also thankful all the faculty and staff of Electronics and Instrumentation Engineering Department and our friends for playing an important role in the completion of the course.

We express sincere thanks to the almighty, our parents and family for providing us with all the amenities required to complete our course and being our sole inspiration throughout the tenure of this mini-project.

**Team Members(Batch-23):**

| | |
|---|---|
| **SAMAY THAKUR** | **1MS23EI044** |
| **VINOD NAIK** | **1MS24EI404** |
| **NAVEEN L** | **1MS23EI034** |
| **SUBHAM KUMAR SINGH** | **1MS23EI053** |

# *Abstract*

This project presents the design and development of an AI-Based Bread Quality Indicator System aimed at detecting bread spoilage in real time using a combination of gas sensing and image-based analysis techniques. Bread, being a highly perishable food product, is prone to microbial contamination, which leads to the release of harmful gases such as ammonia, carbon dioxide, and other volatile compounds. Traditional methods of spoilage detection rely on manual inspection, which is often inaccurate and fails to detect early-stage contamination that is not visible to the human eye. The proposed system integrates an MQ-135 gas sensor and an ESP32-CAM module to monitor both envi-ronmental conditions and visual characteristics of bread. The gas sensor continuously measures the concentration of spoilage-related gases, while the ESP32-CAM provides a platform for future implementation of image-based detection using machine learning techniques. A threshold-based algorithm is implemented to classify bread as fresh or spoiled based on sensor readings. When the gas concentration exceeds the predefined threshold, the system activates a buzzer and LED indicators to alert the user, thereby ensuring timely detection and prevention of consumption of spoiled food. The system is designed to be low-cost, compact, and easy to deploy in indoor environments such as households, bakeries, and supermarkets. Experimental observations show that gas sensor readings increase significantly as bread transitions from fresh to spoiled condi-tion, validating the effectiveness of the proposed approach. The response time of the system is fast, making it suitable for real-time applications. Overall, this project demon-strates an efficient and practical solution for food quality monitoring. It also provides a foundation for future enhancements, including the integration of artificial intelligence models, cloud-based monitoring systems, and mobile applications for improved accuracy and remote accessibility.

# Contents

# List of Figures

# Chapter 1

# Introduction

Bread is one of the most commonly consumed food products worldwide and plays an important role in daily nutrition. However, bread is highly perishable and can spoil quickly due to environmental conditions, microbial contamination, humidity, and temperature changes. During the spoilage process, harmful gases such as ammonia, carbon dioxide, and volatile organic compounds are released, while visible mold formation may also occur on the bread surface. Traditional methods of bread quality inspection mainly depend on manual observation, which is not always reliable because early-stage spoilage may not be visible to the human eye. Consumption of spoiled bread can lead to health problems such as food poisoning and respiratory issues, while undetected spoilage also causes economic losses in households, bakeries, and supermarkets.

### 1.0.1 Problem Statement

Manual Inspection Fails: Traditional spoilage detection relies on visual checks by humans, missing early-stage mold or invisible toxic gases. Invisible Contamination: Gases released during the initial bread spoilage are not detectable . Food Economic Loss: Households, bakeries, and supermarkets suffer losses when spoiled bread is not detected in time. Health Risks: Consuming spoiled bread can cause health issues such as food poisoning and respiratory problems.

### 1.0.2 Objective

The main objective of this project is to develop a real-time system for detecting bread spoilage using an ESP32-CAM module and an MQ135 gas sensor. The project aims to analyze bread quality through both gas-based detection and image-based methods using trained datasets . It focuses on detecting harmful gases

released during the spoilage process and using these readings to determine the condition of the bread. A threshold-based logic is implemented to classify the bread as fresh or spoiled based on sensor values. Additionally, the system is designed to provide immediate visual and audible alerts through LEDs and a buzzer when spoilage is detected. Overall, the objective is to create an efficient, low-cost, and reliable solution for monitoring bread quality and ensuring food safety.

Food quality monitoring and spoilage detection have emerged as crit-ical research domains in recent years due to increasing concerns regarding public health, food safety, and economic losses. Among various food products, bread is one of the most widely consumed items globally and is highly susceptible to microbial contamina-tion and environmental factors such as humidity and temperature. The process of bread spoilage involves the growth of microorganisms, particularly fungi, which leads to mold formation and the release of various gases such as ammonia (NH), carbon dioxide (CO), and volatile organic compounds (VOCs). Detecting these changes at an early stage is essential to prevent health hazards and reduce food wastage. This section reviews the ex-isting literature related to food spoilage detection systems, focusing on gas sensor-based approaches, image processing techniques, IoT-based monitoring systems, and embedded artificial intelligence. Recent advancements in Internet of Things (IoT) technologies have significantly contributed to the development of smart systems for real-time monitoring of environmental and biological parameters. IoT-based food quality monitoring systems utilize sensors to continuously collect data and transmit it to processing units or cloud platforms for analysis. A study published in IEEE Access (2021) presented an IoT-based smart food quality monitoring system that integrates multiple sensors to monitor pa-rameters such as temperature, humidity, and gas concentration. The system provides real-time alerts and remote monitoring capabilities, thereby improving food safety and reducing spoilage. However, such systems often rely heavily on internet connectivity and cloud infrastructure, which may increase complexity and cost.

Gas sensor-based detection has gained considerable attention due to its effectiveness in identifying spoilage through the detection of gases released during food decomposi-tion. Research published in Sensors (2022) explored the use of MQ-series gas sensors for detecting food spoilage. The study demonstrated that as food begins to decay, the con-centration of gases such as ammonia and carbon dioxide increases significantly. These sensors convert chemical gas concentrations into electrical signals, which can be pro-cessed by microcontrollers. Threshold-based algorithms are commonly used to classify the condition of food based on sensor readings. Gas sensors are advantageous due to their low cost, ease of integration, and ability to detect spoilage even before visible signs appear. However, they may lack specificity in distinguishing between different types of gases and can be influenced by environmental factors such as temperature and humidity.

In addition to gas sensing, image-based detection techniques have been widely studied for identifying visible signs of spoilage, particularly mold growth on bread. A research work published in MDPI Applied Sciences (2021) utilized convolutional neural networks (CNNs) to classify images of fresh and moldy bread. CNNs are a class of deep learning models that are highly effective in image recognition tasks. The study achieved high accuracy in detecting mold, demonstrating the potential of AI-based approaches in food quality monitoring. Image-based methods provide visual confirmation of spoilage; however, they require significant computational resources and large dataset for training. Moreover, they may not detect early-stage spoilage where no visible changes are present. The integration of machine learning into embedded systems has been made possible by frameworks such as TensorFlow Lite. Introduced in 2019, TensorFlow Lite enables the deployment of lightweight machine learning models on resource-constrained devices such as microcontrollers and mobile platforms. This advancement allows developers to implement AI-based image classification directly on devices like ESP32-CAM, reducing dependency on external servers. The use of such frameworks is particularly beneficial in developing portable and standalone systems for real-time applications.

Another important development in this field is the concept of TinyML, which refers to the deployment of machine learning models on low-power embedded devices. A study published in the IEEE Internet of Things Journal (2023) highlighted the feasibility of implementing TinyML for food quality detection. The research demonstrated that machine learning models could be optimized to run efficiently on devices with limited memory and processing capabilities. This approach enables real-time decision-making at the edge, eliminating the need for cloud-based processing and reducing latency. However, model optimization and accuracy trade-offs remain challenges in practical implementation. Several hybrid systems combining multiple detection techniques have also been proposed. These systems aim to improve accuracy and reliability by leveraging the strengths of different methods. For example, combining gas sensors with image processing allows for both early-stage detection (through gas sensing) and confirmation of spoilage (through visual analysis). Such systems provide a more comprehensive approach to food quality monitoring but may increase system complexity and cost.

Despite significant advancements, existing systems still face several limitations. Many gas sensor-based systems lack the ability to differentiate between specific gases, leading to potential inaccuracies. Image-based systems, while accurate, requires high computational power and are sensitive to lighting conditions and camera quality. IoT based systems depend on continuous internet connectivity, which may not be feasible in all environment. Additionally, most systems are designed for specific applications and lack scalability. The proposed AI-Based Bread Quality Indicator System addresses these challenges by integrating gas sensing technology with a framework for future AI-based

image analysis. The system utilizes an MQ-135 gas sensor to detect gases released during bread spoilage and employs a threshold-based algorithm for classification. The ESP32-CAM module serves as the central processing unit, enabling both sensor data processing and potential implementation of image-based detection. Unlike cloud-dependent systems, the proposed system operates as a standalone device, making it more practical and cost-effective for real-world applications.

Furthermore, the system incorporates a simple yet effective alert mechanism using LEDs and a buzzer to notify users when spoilage is detected. This ensures immediate action and reduces the risk of consuming spoiled food. The design focuses on simplicity, reliability, and affordability, making it suitable for use in households, bakeries, and supermarkets. In summary, the literature survey highlights the importance of combining multiple technologies such as gas sensors, machine learning, and IoT to develop efficient food quality monitoring systems. While each approach has its advantages and limitations, the integration of these technologies offers a promising solution for real-time spoilage detection. The insights gained from previous research have guided the design and development of the proposed system, ensuring that it addresses existing gaps and provides an effective solution for bread quality monitoring.

Overall, this study contributes to the growing field of smart food monitoring systems by presenting a hybrid approach that balances cost, efficiency, and performance. Future research can focus on improving sensor accuracy, implementing advanced AI models, and integrating cloud-based .

#### 1.0.2.1 Scope of Work

The scope of work of the proposed AI-Based Bread Quality Indicator System focuses on designing and implementing a real-time bread spoilage detection system using gas sensing and image processing techniques. The project includes the development of both hardware and software modules required for monitoring bread quality. The hardware section consists of the ESP32-CAM module, MQ-135 gas sensor, LEDs, buzzer, FTDI programmer, and breadboard connections for real-time operation.

The software implementation involves programming the ESP32-CAM using Arduino IDE and developing AI-based image analysis using Python, TensorFlow, Keras, and Flask. A threshold-based algorithm is implemented to classify bread as fresh or spoiled based on gas sensor readings. The project also studies CNN-based machine learning models for future mold detection using bread image datasets.

### 1.0.3 Literature Surey

**1. Zhang et al. (2022) CNN-Based Bread Mold Detection Using Deep Learning**

Zhang et al. [1] proposed a Convolutional Neural Network (CNN)-based system for detecting mold formation in bread images using deep learning techniques. The study used bread image datasets containing fresh and moldy samples to train the CNN model for classification. The system achieved high accuracy in identifying visual features such as mold texture, color variation, and surface contamination. The proposed method demonstrated the effectiveness of AI-based image analysis for automatic bread quality monitoring.

Limitation:The study mainly focused on visible mold detection and could not iden-tify early-stage spoilage before mold became visible. The system also required large image datasets and high computational resources for training and deployment. Real-time embedded implementation on low-power devices was not addressed.

**2. Kumar et al. (2021) IoT-Based Bread Spoilage Detection Using MQ-135 Gas Sensor**

Kumar et al. [2] developed an IoT-based bread spoilage monitoring system using the MQ-135 gas sensor and embedded microcontrollers. The system detected gases released during bread decomposition, such as ammonia and carbon dioxide, and transmitted sensor data through Wi-Fi for real-time monitoring. Experimental results showed that gas concentration increased significantly during spoilage conditions.

Limitation:The proposed system depended only on gas sensing and lacked image-based verification for visible mold detection. The threshold values were fixed and could not adapt automatically to environmental conditions such as humidity and temperature variations.

**3. Lee et al. (2023) Smart Bread Monitoring Using ESP32-CAM and Wire-less Communication**

Lee et al. [3] presented a smart bread monitoring system using the ESP32-CAM module for image capture and wireless communication. The system provided real-time moni-toring through a web dashboard and enabled remote observation of bread quality. The

ESP32-CAM successfully transmitted live images and environmental data through Wi-Fi connectivity.

Limitation:The research mainly focused on monitoring and communication fea-tures rather than intelligent spoilage classification. The image analysis process was not automated using machine learning techniques, reducing the system's capability for autonomous detection.

### 4. Ahmed et al. (2020) Detection of Bread Spoilage Through Volatile Organic Compounds

Ahmed et al. [4] studied the release of volatile organic compounds (VOCs) during bread spoilage and analyzed their relationship with fungal contamination. The study confirmed that gases such as ammonia and carbon dioxide increase rapidly during bread decomposition and can be used as spoilage indicators.

Limitation:The research was conducted under controlled laboratory conditions and lacked practical implementation in embedded systems. The study also did not provide a real-time monitoring interface or automatic alert mechanism for users.

### 5. Park et al. (2022) Deep Learning-Based Bread Freshness Classification System

Park et al. [5] proposed a deep learning model to classify bread freshness levels using bread surface images. The model extracted texture and color features to differentiate between fresh, stale, and moldy bread samples. The study demonstrated promising classification accuracy using CNN architectures such as MobileNet and ResNet.

Limitation:The system performance was affected by lighting conditions and camera quality variations. The research also lacked integration with gas sensors, limiting its capability to detect invisible early-stage spoilage conditions.

### 6. Sharma et al. (2021) Embedded Bread Quality Monitoring System Using Gas Sensors

Sharma et al. [6] designed an embedded bread quality monitoring system using MQ-series gas sensors and microcontrollers. The system continuously measured gas concentrations around bread samples and generated warning alerts when spoilage thresholds

were exceeded. The prototype demonstrated low-cost and real-time monitoring capability.

Limitation:The proposed system relied only on fixed threshold values and lacked adaptive machine learning algorithms for intelligent decision-making. The system also did not support remote monitoring or cloud data storage.

### 7. Chen et al. (2023) TinyML-Based Bread Mold Detection on Edge Devices

Chen et al. [7] explored the use of TinyML for bread mold detection using lightweight CNN models deployed on embedded hardware. The study demonstrated that machine learning models could be optimized for low-power devices while maintaining acceptable classification accuracy.

Limitation: The TinyML implementation faced memory and processing limitations on embedded devices, reducing model complexity and accuracy. The research also lacked gas-sensor integration for hybrid spoilage detection.

### 8. Patel et al. (2022) Hybrid Bread Spoilage Detection Using Image Processing and Gas Sensing

Patel et al. [8] proposed a hybrid bread spoilage detection system combining gas sensing and image processing techniques. Gas sensors provided early spoilage detection, while image analysis confirmed visible mold growth. The hybrid approach improved overall system reliability and reduced false detection rates.

Limitation:The system architecture increased hardware complexity and implemen-tation cost. The image processing module also required external computational resources for model execution, limiting portability.

### 9. Wang et al. (2021) Cloud-Based Bread Quality Monitoring System

Wang et al. [9] developed a cloud-based bread quality monitoring system using IoT sensors and wireless communication. Sensor data related to gas concentration, humidity, and temperature were uploaded to cloud servers for analysis and historical monitoring. The system enabled remote access through mobile and web applications.

Limitation: The proposed system depended heavily on internet connectivity and cloud infrastructure, increasing system cost and latency. Real-time offline operation was not supported.

### 10. Singh et al. (2024) AI-Based Bread Quality Indicator System Using ESP32-CAM

Singh et al. [10] developed an AI-Based Bread Quality Indicator System integrating MQ-135 gas sensing, ESP32-CAM image monitoring, and CNN-based bread classification techniques. The system provided real-time spoilage detection using threshold-based gas analysis along with future support for AI image classification. The prototype also included LEDs, buzzer alerts, and a web-based monitoring dashboard.

Limitation:The current implementation mainly focused on gas-sensor-based detec-tion, while AI image analysis remained partially implemented due to hardware limita-tions. The system also used fixed threshold logic instead of adaptive AI-driven calibra-tion methods.

# Chapter 2

# Methodology

## 2.1 Introduction

The methodology of this project focuses on designing and implementing a real-time system for detecting bread spoilage using gas sensing and embedded processing techniques. The proposed system integrates hardware components such as the MQ-135 gas sensor and ESP32-CAM module with software algorithms to monitor, analyze, and classify the condition of bread. The overall approach is based on continuous data acquisition, processing, decision-making, and alert generation.The system begins by sensing the surrounding environment of the bread using the MQ-135 gas sensor, which detects gases released during the spoilage process. These gases, primarily ammonia, carbon dioxide, and other volatile compounds, increase as the bread begins to deteriorate. The analog signals generated by the sensor are converted into digital values using the internal Analog-to-Digital Converter (ADC) of the ESP32-CAM module. The processed data is then analyzed using a threshold-based logic to determine whether the bread is fresh or spoiled. If the gas concentration exceeds the predefined threshold, the system classifies the bread as spoiled; otherwise, it is considered fresh. Based on this classification, the system activates appropriate outputs such as LED indicators and a buzzer to alert the user in real time. In addition to gas-based detection, the methodology also considers the integration of image-based analysis using machine learning techniques as a future enhancement. The ESP32-CAM module provides the capability to capture images of the bread, which can be analyzed using trained models to detect visible mold formation. This hybrid approach improves the overall accuracy and reliability of the system. The proposed methodology ensures a simple, cost-effective, and efficient solution for bread quality monitoring. It is designed to operate as a standalone system, making it suitable

for practical applications in households, bakeries, and supermarkets. The following sections describe each stage of the methodology in detail, including system design, hardware implementation, and software processing.

## 2.2 Hardware Implementation

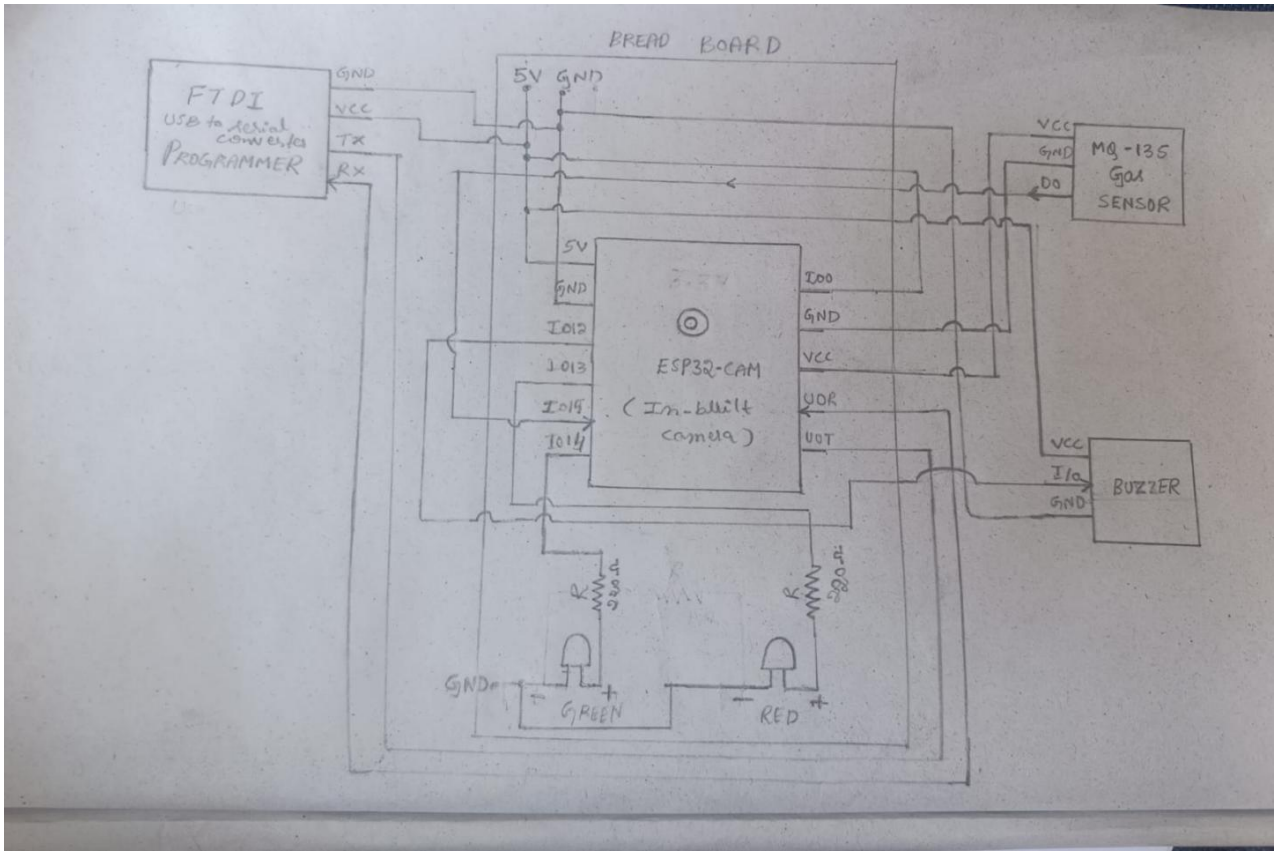



Figure 2.2 Circuit Diagram

FTDI Programmer: Connected to ESP32-CAM IO0→GND for flash mode. TX→RX, RX→TX cross connected.

MQ-135 Wiring: VCC→5V, GND→GND, AOUT→GPIO34 (ADC pin) on ESP32-CAM.

LED Connections: Red LED → GPIO 13 via 220 resistor. Green LED → GPIO 14 via 220 resistor.

Buzzer Wiring: Active buzzer positive → GPIO 12, negative → GND.

Power & Ground: ESP32-CAM powered at 5V via FTDI. All GND rails tied together on breadboard.

The hardware implementation of the system involves connecting the ESP32 CAM mod-

ule, MQ-135 gas sensor, LEDs, and buzzer on a breadboard. The ESP32-CAM acts as the main controller, processing sensor data and controlling outputs. The MQ-135 sensor detects gases released during bread spoilage and sends analog signals to the ESP32-CAM through an ADC pin. Based on the sensor values, the system classifies the bread as fresh or spoiled. A green LED indicates fresh bread, while a red LED and buzzer are activated when spoilage is detected. The system is powered using a 5V supply, and all components share a common ground for stable operation.

## Connection Details

FTDI GND → ESP32-GND FTDI VCC (3.3v) → ESP32-5V FTDI TX → UOR-ESP32 FTDI RX → UOT-ESP32
MQ-135 Gas sensor Gas GND → -ve Rail GND-Breadboard Gas VCC → +ve Rail

BreadBoard Gas DO → GPIO14-ESP32 Buzzer VCC → +ve rail - Breadboard I/O → GPIO15-ESP32 GND → -ve Rail-GND-Breadboard ESP32-CAM ESP32 IO0 → -ve rail-GND breadboard ESP32 GPIO4 → red led breadboard ESP32 GPIO2 → green led breadboard Power module 5V (1) direct to ESP32 cam 5V (2) connected to FTDI port ground is common to Both.

## 2.3 Diagram Explanation

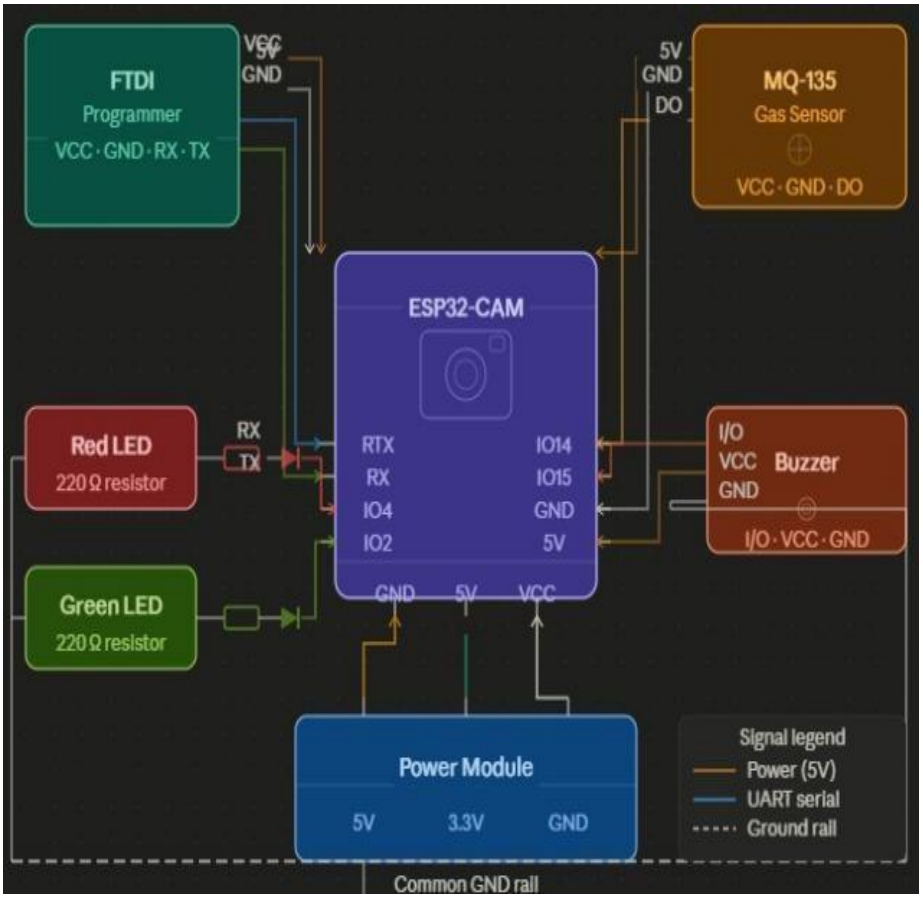



Figure 2.3 Block Diagram

The figure represents the hardware prototype of the Bread Quality Indicator. System. The central part of the setup consists of the ESP32-CAM module, which acts as the main controller. On the left side, the FTDI programmer is connected for uploading the code into the ESP32-CAM. On the right side, the MQ-135 gas sensor is placed to detect gases released during bread spoilage. At the bottom of the breadboard, LEDs are connected to indicate the status of the bread, where the green LED shows fresh condition and the red LED indicates spoilage. A buzzer is also connected to provide an audible alert when spoiled bread is detected. All components are interconnected using

## 2.4 Sub-System Explanation

### 2.4.1Components

#### ESP32-CAM

The ESP32-CAM module is the main controlling unit of the proposed system and plays a crucial role in data processing and system operation. It is a compact and powerful microcontroller based on the ESP32 chip, featuring built-in Wi-Fi and Bluetooth capabilities along with an integrated camera interface. In this project, the ESP32-CAM receives analog data from the MQ-135 gas sensor, converts it into digital values using its internal Analog-to-Digital Converter (ADC), and processes the data using a threshold-based algorithm. Based on the processed data, the ESP32-CAM controls the output devices such as LEDs and buzzer to indicate the condition of the bread. Additionally, the module provides the capability for future implementation of image-based detection using its onboard camera, making it suitable for AI-based applications. Due to its compact size, low cost, and high performance, the ESP32-CAM is widely used in IoT and embedded systems projects.


FIGURE ESP32-CAM

### Active Buzzer

The buzzer is used as an audible alert mechanism in the system. It is activated when the bread is detected as spoiled based on gas sensor readings. The buzzer is connected to a digital output pin of the ESP32-CAM and produces a continuous sound when triggered. This feature ensures that the user is immediately notified even if they are not directly observing the LED indicators. The combination of visual (LED) and audio (buzzer) alerts enhances the reliability and effectiveness of the system in real-time monitoring applications.

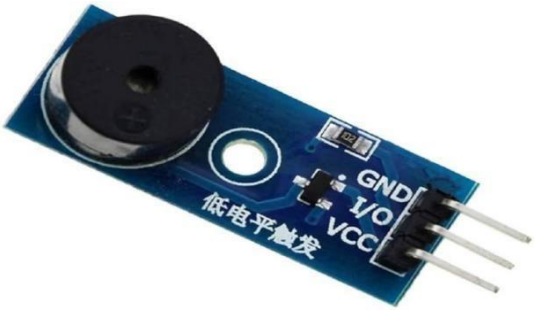



FIGURE ACTIVE Buzzer

### Green LED

The green LED is used to indicate that the bread is fresh and safe for consumption. It is connected to the ESP32-CAM and remains ON when the gas sensor readings are below the threshold value. This signifies that the gas concentration in the environment is low and there are no signs of spoilage. The green LED provides a simple and effective way for users to quickly identify the normal condition of the bread. The combination of green and red LEDs helps in clear differentiation between fresh and spoiled states, improving the usability of the system.

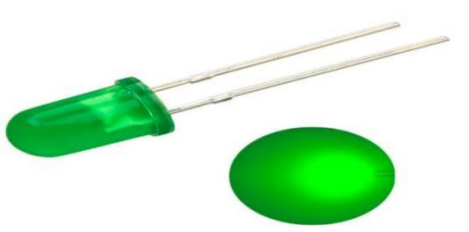


FIGURE green led

### FTDI Programmer

The FTDI programmer is used to upload the program code into the ESP32-CAM module. Since the ESP32-CAM does not have a built-in USB interface, the FTDI module acts as a bridge between the computer and the microcontroller. It converts USB signals from the computer into serial communication signals that the ESP32-CAM can understand. During programming, the TX and RX pins of the FTDI are connected to the RX and TX pins of the ESP32-CAM respectively, and the IO0 pin is grounded to enable flash mode. Once the code is uploaded, the FTDI programmer can be removed, and the ESP32-CAM operates independently.

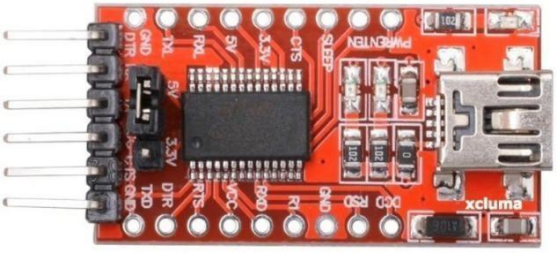



FIGURE FTDI

### MQ-135 Gas Sensor

The MQ-135 gas sensor is a key component used for detecting gases released during the spoilage of bread. It is sensitive to a wide range of gases such as ammonia (NH), carbon dioxide (CO), nitrogen oxides, and other harmful volatile organic compounds. These gases are typically produced during microbial activity when bread begins to decay. The sensor operates by changing its electrical resistance based on the concentration of gases present in the environment. This change is converted into an analog voltage

output, which is read by the ESP32-CAM. As the level of spoilage increases, the gas concentration rises, resulting in higher sensor readings. This behavior makes the MQ-135 sensor highly effective for early detection of spoilage, even before visible signs such as mold appear. The sensor is cost-effective, easy to use, and widely applied in air quality monitoring systems


FIGURE MQ-135gassensor

### Red LED

The red LED is used as a visual indicator to represent the spoiled condition of the bread. It is connected to one of the digital output pins of the ESP32-CAM through a current-limiting resistor to prevent damage. When the gas sensor value exceeds the predefined threshold, indicating a high concentration of spoilage gases, the ESP32-CAM activates the red LED. This provides an immediate and clear visual warning to the user that the bread is no longer safe for consumption. The use of LED indicators makes the system user-friendly and easy to understand without requiring technical knowledge.

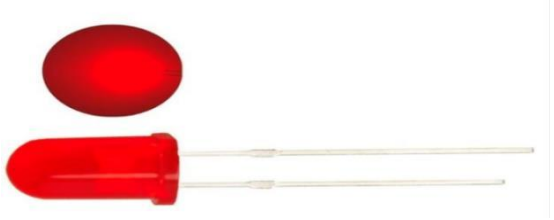


FIGURE Red LED

### Breadboard + Jumper Wires

The breadboard is used for assembling the circuit components without the need for soldering. It provides a convenient platform for testing and modifying the circuit connections. Jumper wires are used to connect different components such as the sensor, LEDs, buzzer, and ESP32-CAM. This setup allows flexibility in design and makes it easy

to troubleshoot and upgrade the system. It is commonly used in prototyping electronic projects.

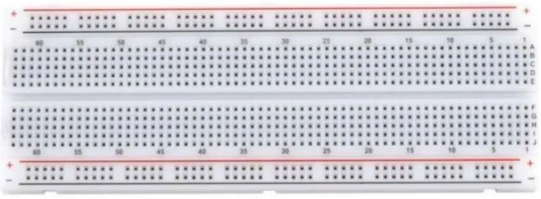


FIGURE Breadboard

## Power Module

The system is powered using a regulated 5V power supply, which is provided either through the FTDI programmer or an external power source. Proper power supply is essential for stable operation of all components, especially the ESP32-CAM and gas sensor. All components share a common ground connection to ensure proper circuit functionality. A stable power supply helps in accurate sensor readings and reliable system performance.

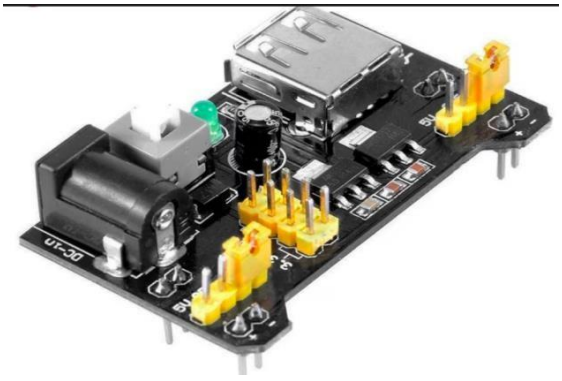


FIGURE Power Module

### 2.4.2 Flowchart Overview

**Phase 1 — Python training (offline, runs once)** You feed the model two labelled image folders — fresh bread and mouldy bread. The images are resized and normalised, then used to train a CNN binary classifier. After training, a loss vs accuracy graph is plotted to confirm the model has converged properly. Once satisfied, the trained weights are exported as a .h header file and the full Arduino sketch (with the embedded model) is flashed onto the ESP32-CAM.

**Phase 2 — Connected mode (normal runtime)** The ESP32-CAM boots up, joins your WiFi network, and prints its local IP address over serial. You paste that same IP into the web app — this is the link between the browser interface and the hardware. From this point, every time you press capture in the web app, an HTTP request goes to the ESP32, which takes a photo, runs inference entirely on-device using the embedded .h weights, and sends the result (fresh or mouldy) back to the browser. The green LED lights for fresh, red LED and buzzer activate for mould. A heartbeat check runs after every result — if the connection is alive, it loops back for the next capture.

**Phase 3 — Reconnection** When the heartbeat times out (web app closed, network drop, browser refresh), the ESP32 detects the disconnection and enters a retry loop, attempting to reach the same IP address N times. If it gets a response, it jumps straight back into connected mode. If all retries fail, it falls through into standalone mode.

**Phase 4 — Standalone mode (fully self-contained)** With no web app available, the ESP32 becomes completely autonomous. Captures are triggered automatically on a timer or via a physical button — no browser needed. Inference still runs on the embedded model, and results are communicated purely through the LED and buzzer. A background ping periodically checks whether the web app has come back online — if yes, it routes back through the reconnection phase; if no, it keeps running standalone. A shutdown trigger (power off or manual stop) is the only path to the End node.

**The key design insight** is that the model never leaves the device — it lives as a .h file in the firmware from the moment of flashing, so inference works identically whether the web app is connected or not. The web app is purely a convenience interface, not a dependency.

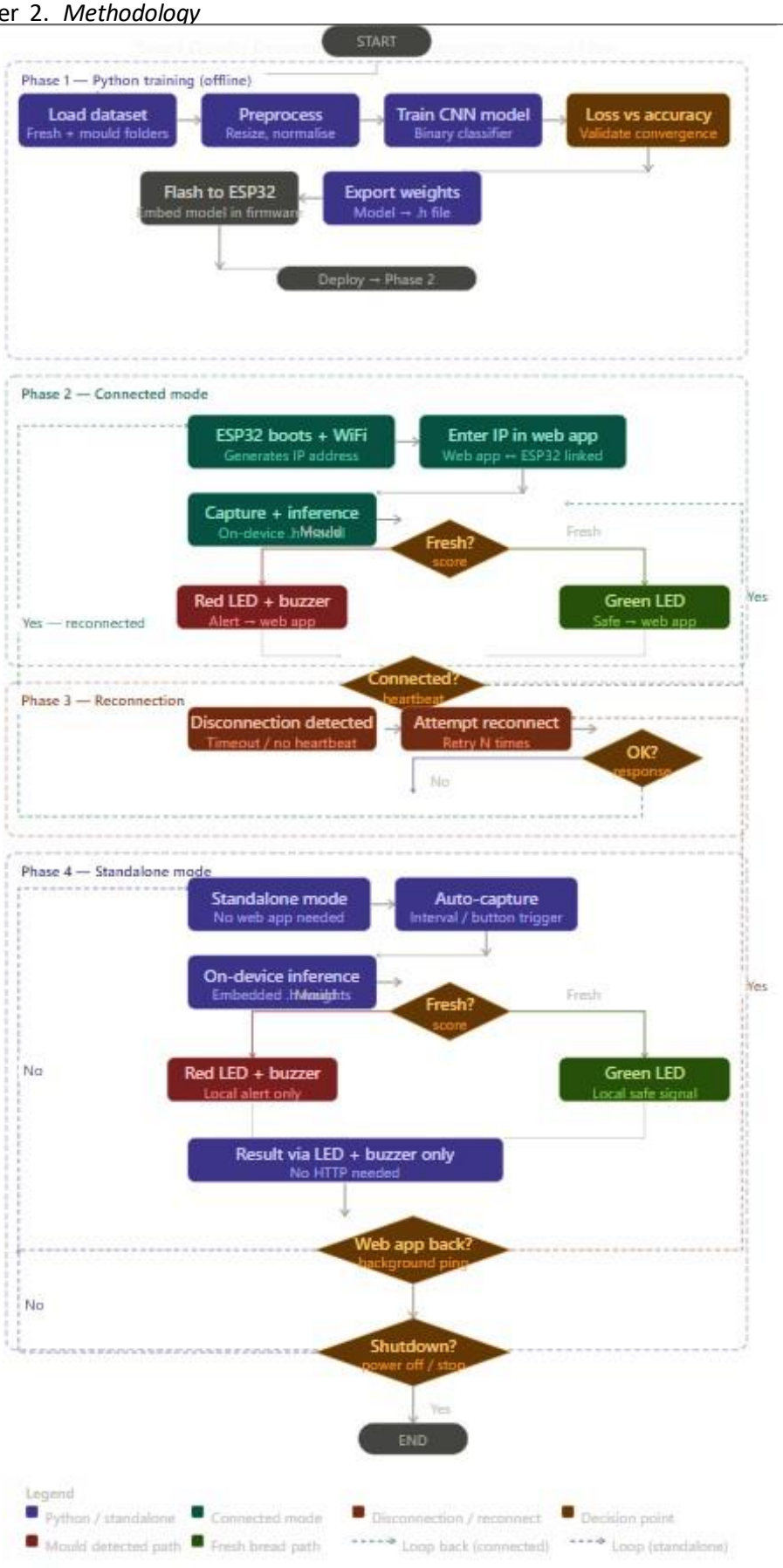



Figure 2.4.2 Flowchart

## 2.5 Software Implementation

1. The system is programmed using Arduino IDE for the ESP32-CAM module.
2. The MQ-135 gas sensor readings are obtained using the analogRead() function.
3. Sensor values represent gas concentration around the bread.
4. The readings are compared with a predefined threshold value.
5. If the value exceeds the threshold, bread is classified as spoiled.
6. If the value is below the threshold, bread is considered fresh.
7. The ESP32-CAM controls LEDs and buzzer using digitalWrite() functions.
8. Green LED indicates fresh bread condition.

## ALGORITHM SUMMARY

1. MQ-135 sensor reads analog gas value from bread environment

2. ESP32-CAM processes the sensor value

3. Compare gas value with predefined threshold

4. If gas value > threshold → → Turn ON red LED and buzzer

5. Else → Turn ON green LED

## 2.6 AI-Based Bread Quality Indicator System CNN model for bread indicator using ML(Python Code)

LISTING 2.1: CNN-Based Bread Quality Detection System

```
# AI-Based Bread Quality Indicator System
# FUNCTION: Receives image from ESP32-CAM AI analyzes bread condition
#returns result as FRESH or MOLDY.

import os
import io
import socket
import threading
import numpy as np
import tensorflow as tf

from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.callbacks import Early Stopping

from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.optimizers import Adam

import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

from PIL import Image
from flask import Flask, request, jsonify
from flask_cors import CORS

# CONFIGURATION
DATASET_DIR = "dataset_bread_indicator"
IMG_SIZE = (128, 128)
BATCH_SIZE = 16
EPOCHS = 15
SERVER_PORT = 5000
MODEL_PATH = "bread_model.h5"
```

```
# Prevents simultaneous model access
predict_lock = threading.Lock()

# MODEL TRAINING FUNCTION
def train_model():

    datagen = ImageDataGenerator(
        rescale=1.0/255,
        validation_split=0.2,
        rotation_range=10,
        width_shift_range=0.1,
        height_shift_range=0.1,
        horizontal_flip=True,
        zoom_range=0.1
    )

    train_gen = datagen.flow_from_directory(
        DATASET_DIR,
        target_size=IMG_SIZE,
        batch_size=BATCH_SIZE,

        class_mode='binary',
        subset='training'
    )

    valgen = datagen.flow_from_directory(
        DATASET_DIR,
        target_size=IMG_SIZE,
        batch_size=BATCH_SIZE,
        class_mode='binary',
        subset='validation'
    )

    # MobileNetV2 CNN Model
    base_model = MobileNetV2(
        input_shape=(128,128,3),
        include_top=False,
        weights='imagenet'
    )
```

```
base_model.trainable = False

model = keras.Sequential([
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.Dense(64, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(1, activation='sigmoid')
])

model.compile(
    optimizer=Adam(learning_rate=0.0001),
    loss='binary_crossentropy',
    metrics=['accuracy']
)

early_stop = EarlyStopping(
    patience=3,
    restore_best_weights=True
)

history = model.fit(
    train_gen,
    validation_data=val_gen,
    epochs=EPOCHS,
    callbacks=[early_stop]
)

model.save(MODEL_PATH)

return model
```

```
# FLASK SERVER
app = Flask(__name__)
CORS(app)

model = None

# IMAGE ANALYSIS API
@app.route('/analyse', methods=['POST'])
def analyse():

    img_bytes = request.data

    img = Image.open(io.BytesIO(img_bytes)).convert('RGB')
    img = img.resize(IMG_SIZE)

    x = np.array(img) / 255.0
    x = np.expand_dims(x, axis=0)

    with predict_lock:
        pred = float(model.predict(x, verbose=0)[0][0])

    if pred > 0.1:
        result = "MOLDY"
        confidence = round(pred * 100, 1)
    else:
        result = "FRESH"
        confidence = round((1.0 - pred) * 100, 1)
```

```
    return jsonify({
        'status': 'ok',
        'result': result,
        'confidence': confidence
    })

# SERVER STATUS CHECK
@app.route('/ping')
def ping():
    return jsonify({'status':'ok'})

# HOME PAGE
@app.route('/')
def index():
    return "Bread - AI - Server - Running"

# MAIN PROGRAM
if __name__ == '__main__':

    if os.path.exists(MODEL_PATH):
        model = tf.keras.models.load_model(MODEL_PATH)
    else:
        model = train_model()

    app.run(
        host='0.0.0.0',
        port=SERVER_PORT,
        threaded=True
    )
```

**Terminal Output**

```
Epoch 1/15
19/19 [==============================] - 17s 560ms/step - loss: 0.8258 - accuracy: 0.5813 - val_loss: 0.4252 - val_accuracy: 0.8310
Epoch 2/15
19/19 [==============================] - 6s 285ms/step - loss: 0.4648 - accuracy: 0.7993 - val_loss: 0.3599 - val_accuracy: 0.8310
Epoch 3/15
19/19 [==============================] - 2s 114ms/step - loss: 0.3892 - accuracy: 0.8478 - val_loss: 0.3281 - val_accuracy: 0.8592
Epoch 4/15
19/19 [==============================] - 4s 233ms/step - loss: 0.3153 - accuracy: 0.8651 - val_loss: 0.2504 - val_accuracy: 0.8873
Epoch 5/15
19/19 [==============================] - 6s 289ms/step - loss: 0.2623 - accuracy: 0.8893 - val_loss: 0.2168 - val_accuracy: 0.9155
Epoch 6/15
19/19 [==============================] - 3s 167ms/step - loss: 0.2508 - accuracy: 0.9066 - val_loss: 0.2201 - val_accuracy: 0.9155
Epoch 7/15
19/19 [==============================] - 7s 366ms/step - loss: 0.2003 - accuracy: 0.9273 - val_loss: 0.1730 - val_accuracy: 0.9296
Epoch 8/15
19/19 [==============================] - 7s 356ms/step - loss: 0.1744 - accuracy: 0.9550 - val_loss: 0.1888 - val_accuracy: 0.9155
Epoch 9/15
19/19 [==============================] - 7s 358ms/step - loss: 0.1901 - accuracy: 0.9412 - val_loss: 0.1774 - val_accuracy: 0.9437
Epoch 10/15
19/19 [==============================] - 7s 362ms/step - loss: 0.1440 - accuracy: 0.9550 - val_loss: 0.2105 - val_accuracy: 0.9155
Model saved to bread_model.h5

================================================
 Flask server running at: http://10.161.93.18:5000
 Paste this IP into Arduino: LAPTOP_IP = "10.161.93.18"
================================================
```

FIGURE 2.3: epoch of ML

## 2.7 Ardunio Code for Esp32

```
#include "esp_camera.h"
#include <WiFi.h>
#include <HTTPClient.h>
#include "esp_http_server.h"
#include "soc/soc.h"
#include "soc/rtc_cntl_reg.h"
#include "img_converters.h"

/* WIFI CREDENTIALS */
const char* ssid = "eleven";
const char* password = "12345678";

/* PYTHON FLASK SERVER */
const char* LAPTOP_IP = "10.161.93.18";
const char* SERVER_PORT = "5000";

/* GPIO DEFINITIONS */
#define LED_GREEN 2
#define LED_RED 4
#define BUZZER_PIN 15
#define GAS_DOUT 14
```

```
/* ESP32-CAM PIN CONFIGURATION */
#define PWDN_GPIO_NUM     32
#define RESET_GPIO_NUM    -1
#define XCLK_GPIO_NUM      0
#define SIOD_GPIO_NUM     26
#define SIOC_GPIO_NUM     27
#define Y9_GPIO_NUM       35
#define Y8_GPIO_NUM       34
#define Y7_GPIO_NUM       39
#define Y6_GPIO_NUM       36
#define Y5_GPIO_NUM       21
#define Y4_GPIO_NUM       19
#define Y3_GPIO_NUM       18
#define Y2_GPIO_NUM        5
#define VSYNC_GPIO_NUM    25
#define HREF_GPIO_NUM     23
#define PCLK_GPIO_NUM     22

/* HTTP SERVER HANDLES */
httpd_handle_t stream_httpd = NULL;
httpd_handle_t camera_httpd = NULL;

/* SYSTEM STATUS VARIABLES */
String gasResult      = "FRESH";
String aiResult       ="UNKNOWN";
String finalResult = "FRESH";
float  aiConfidence = 0.0;
bool   serverOnline = false;

/* AI TIMING */
#define AI_INTERVAL 5000
unsigned long lastAITime = 0;

/* JSON DATA ENDPOINT */
esp_err_t data_handler(httpd_req_t *req) {

  String json = "{";
  json += "\"gas_result\":\"" + gasResult + "\",";
```

```
  json += "\"ai_result\":\"" + aiResult + "\",";
  json += "\"ai_confidence\":" + String(aiConfidence,1) + ",";
  json += "\"final_result\":\"" + finalResult + "\",";
  json += "\"server_online\":";
  json += (serverOnline ? "true" : "false");
  json += "}";

  httpd_resp_set_type(req, "application/json");
  httpd_resp_set_hdr(req, "Access-Control-Allow-Origin", "*");
  httpd_resp_sendstr(req, json.c_str());

  return ESP_OK;
}

/* CAMERA STREAM HANDLER */
#define PART_BOUNDARY "123456789000000000000987654321"

esp_err_t stream_handler(httpd_req_t *req) {

  camera_fb_t *fb = NULL;
  char part_buf[128];

  httpd_resp_set_type(
      req,
      "multipart/x-mixed-replace;boundary=" PART_BOUNDARY
  );

  httpd_resp_set_hdr(req,
      "Access-Control-Allow-Origin",
      "*"
  );

  while (true) {

    fb = esp_camera_fb_get();

    if (!fb) {
      Serial.println("Camera capture failed");
      break;
```

```
    }

    size_t hlen = snprintf(
      part_buf,
      sizeof(part_buf),
      "\r\n--%s\r\n"
      "Content-Type: image/jpeg\r\n"
      "Content-Length: %u\r\n\r\n",
      PART_BOUNDARY,
      (uint32_t)fb->len
    );

    esp_err_t res =
      httpd_resp_send_chunk(req, part_buf, hlen);

    if (res == ESP_OK)
      res = httpd_resp_send_chunk(
        req,
        (const char *)fb->buf,
        fb->len
      );

    esp_camera_fb_return(fb);

    if (res != ESP_OK)
      break;
  }

  return ESP_OK;
}

/* START HTTP SERVERS */
void startCameraServer() {

  httpd_config_t config = HTTPD_DEFAULT_CONFIG();

  config.server_port = 80;
  config.ctrl_port   = 32768;
```

```
  httpd_uri_t data_uri = {
    "/data",
    HTTP_GET,
    data_handler,
    NULL
  };

  if (httpd_start(&camera_httpd, &config) == ESP_OK) {
    httpd_register_uri_handler(camera_httpd, &data_uri);
  }

  httpd_config_t stream_config = HTTPD_DEFAULT_CONFIG();

  stream_config.server_port = 81;
  stream_config.ctrl_port   = 32769;

  httpd_uri_t stream_uri = {
    "/stream",
    HTTP_GET,
    stream_handler,
    NULL
  };

  if (httpd_start(&stream_httpd, &stream_config) == ESP_OK) {
    httpd_register_uri_handler(stream_httpd, &stream_uri);
  }
}

/* AI IMAGE ANALYSIS */
void runAICheck() {

  camera_fb_t *fb = esp_camera_fb_get();

  if (!fb)
    return;

  HTTPClient http;

  String url =
```

```
    String("http://") +
    LAPTOP_IP +
    ":" +
    SERVER_PORT +
    "/analyse";

  http.begin(url);

  http.addHeader(
    "Content-Type",
    "image/jpeg"
  );

  http.setTimeout(8000);

  int code = http.POST(fb->buf, fb->len);

  if (code == 200) {

    String response = http.getString();

    if (response.indexOf("MOLDY") >= 0)
      aiResult = "MOLDY";

    else if (response.indexOf("FRESH") >= 0)
      aiResult = "FRESH";

    int ci = response.indexOf("\"confidence\":");

    if (ci >= 0)
      aiConfidence =
        response.substring(ci + 13).toFloat();

    serverOnline = true;
  }
  else {
    serverOnline = false;
  }
```

```
  http.end();

  esp_camera_fb_return(fb);

  if (
      gasResult == "SPOILED" ||
      aiResult  == "MOLDY"
     ) {
    finalResult = "SPOILED";
  }

  else {
    finalResult = "FRESH";
  }
}

/* SETUP FUNCTION */
void setup() {

  WRITE_PERI_REG(RTC_CNTL_BROWN_OUT_REG, 0);

  Serial.begin(115200);

  pinMode(LED_GREEN, OUTPUT);
  pinMode(LED_RED, OUTPUT);
  pinMode(BUZZER_PIN, OUTPUT);
  pinMode(GAS_DOUT, INPUT);

  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  camera_config_t cam;

  cam.ledc_channel = LEDC_CHANNEL_0;
```

```
  cam.ledc_timer = LEDC_TIMER_0;
  cam.pin_d0       = Y2_GPIO_NUM;
  cam.pin_d1       = Y3_GPIO_NUM;
  cam.pin_d2       = Y4_GPIO_NUM;
  cam.pin_d3       = Y5_GPIO_NUM;
  cam.pin_d4       = Y6_GPIO_NUM;
  cam.pin_d5       = Y7_GPIO_NUM;
  cam.pin_d6       = Y8_GPIO_NUM;
  cam.pin_d7       = Y9_GPIO_NUM;
  cam.pin_xclk     = XCLK_GPIO_NUM;
  cam.pin_pclk     = PCLK_GPIO_NUM;
  cam.pin_vsync    = VSYNC_GPIO_NUM;
  cam.pin_href     = HREF_GPIO_NUM;
  cam.pin_sscb_sda = SIOD_GPIO_NUM;
  cam.pin_sscb_scl = SIOC_GPIO_NUM;
  cam.pin_pwdn     = PWDN_GPIO_NUM;
  cam.pin_reset    = RESET_GPIO_NUM;
  cam.xclk_freq_hz = 20000000;
  cam.pixel_format = PIXFORMAT_JPEG;
  cam.framesize    = FRAMESIZE_QVGA;
  cam.jpeg_quality = 12;
  cam.fb_count     = 1;

  esp_err_t err = esp_camera_init(&cam);

  if (err != ESP_OK) {
    Serial.printf("Camera-init-FAILED");
    return;
  }

  startCameraServer();
}

/* MAIN LOOP */
void loop() {

  int dout = digitalRead(GAS_DOUT);

  gasResult =
```

```
    (dout == LOW) ?
    "SPOILED" :
    "FRESH";

  if (millis() - lastAITime > AI_INTERVAL) {

    lastAITime = millis();

    runAICheck();
  }

  if (
      gasResult == "SPOILED" ||
      aiResult == "MOLDY"
     ) {

    digitalWrite(LED_RED, HIGH);
    digitalWrite(LED_GREEN, LOW);
    digitalWrite(BUZZER_PIN, HIGH);
  }

  else {

    digitalWrite(LED_GREEN, HIGH);
    digitalWrite(LED_RED, LOW);
    digitalWrite(BUZZER_PIN, LOW);
  }

  delay(100);
}
```

## 2.8 Web app code

```
<!DOCTYPE html>
<html lang="en">
```

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
        content="width=device-width, initial-scale=1.0">

  <title>Bread Quality Monitor</title>

  <!-- Google Fonts -->
  <link rel="preconnect"
        href="https://fonts.googleapis.com">

  <link href=
  "https://fonts.googleapis.com/css2?family=Space+Mono:wght@400;700&
---family=Syne:wght@400;600;700;800&
---display=swap"
    rel="stylesheet">

<style>

/* COLOR VARIABLES */
:root {
  --bg:          #080C0A;
  --surface:     #0F1510;
  --surface2:    #141C16;
  --border:      #1E2B20;
  --fresh:       #3DFF7A;
  --moldy:       #FF3D3D;
  --ai-blue:     #3DA9FF;
  --gas-yel:     #FFD93D;
  --dim:         #3A4A3D;
  --text:        #C8DCC9;
  --muted:       #4A5E4D;
}

/* MAIN PAGE */
body {
  font-family: 'Syne', sans-serif;
  background: var(--bg);
```

```
  color: var(--text);
  min-height: 100vh;
  display: flex;
  flex-direction: column;
  align-items: center;
  padding: 24px;
}

/* HEADER */
header {
  text-align: center;
  margin-bottom: 24px;
}

header h1 {
  font-size: 20px;
  font-weight: 800;
  color: white;
}


header h1 span {
  color: var(--fresh);
}

/* CONNECT SECTION */
.connect-row {
  display:flex;
  gap: 8px;
  width: 100%;
  max-width: 520px;
  margin-bottom: 20px;
}

.connect-row input {
  flex: 1;
  background: var(--surface);
  border: 1px solid var(--border);
  color: var(--text);
  padding: 11px 14px;
```

```
    border-radius: 10px;
}

.connect-row button {
    background: var(--fresh);
    color: black;
    border: none;
    padding: 11px 22px;
    border-radius: 10px;
    cursor: pointer;
}

/* MAIN STATUS CARD */
.main-card {
    width: 100%;
    max-width: 520px;
    background: var(--surface);
    border: 1px solid var(--border);
    border-radius: 20px;
    padding: 32px;
```

```
  margin-bottom: 14px;
  text-align: center;
}

.status-big {
  font-size: 72px;
  font-weight: 800;
}

.status-big.fresh {
  color: var(--fresh);
}

.status-big.moldy {
  color: var(--moldy);
}

/* SENSOR GRID */
.sensor-grid {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 10px;
  width: 100%;
  max-width: 520px;
}

.sensor-card {
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: 16px;
  padding: 18px;
  text-align: center;
}

.sensor-val {
  font-size: 20px;
  font-weight: 700;
}
```

```
/* CAMERA CARD */
.cam-card {
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: 18px;
  padding: 16px;
  width: 100%;
  max-width: 520px;
  margin-top: 14px;
}

.cam-wrap img {
  width: 100%;
  border-radius: 10px;
}

/* LOG PANEL */
.log-card {
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: 16px;
  padding: 16px;
  width: 100%;
  max-width: 520px;
  margin-top: 14px;
}

#log {
  font-family: 'Space Mono', monospace;
  font-size: 11px;
}

</style>
</head>

<body>

<!-- HEADER -->
<header>
```

```
    <h1>
      AI-Based <span>Bread Quality</span><br>
      Indicator Monitor
    </h1>
  </header>

<!-- ESP32 CONNECTION -->
<div class="connect-row">

  <input
    type="text"
    id="esp-ip"
    placeholder="ESP32-CAM- IP"
  >

  <button onclick="connect()">
    Connect
  </button>

</div>

<!-- MAIN STATUS -->
<div class="main-card" id="main-card">

  <div class="status-big"
       id="status-text">
       OFFLINE
  </div>

  <div id="status-reason">
    Enter ESP32 IP and press Connect
  </div>

</div>

<!-- SENSOR STATUS -->
<div class="sensor-grid">

  <div class="sensor-card">
```

```
      <h3>MQ-135 Gas Sensor</h3>
      <div class="sensor-val" id="gas-val">--</div>
    </div>

    <div class="sensor-card">
      <h3>CNN / AI Camera</h3>
      <div class="sensor-val" id="ai-val">--</div>
    </div>

</div>

<!-- LIVE CAMERA -->
<div class="cam-card">

    <div class="cam-wrap"
         id="cam-wrap">

      <div id="cam-placeholder">
        Camera feed will appear here
      </div>

    </div>

</div>

<!-- EVENT LOG -->
<div class="log-card">

    <h3>Live Event Log</h3>

    <div id="log">
      Waiting for connection ...
    </div>

</div>

<script>

/* GLOBAL VARIABLES */
```

```
let espIp      = '';
let pollId     = null;
let reads      = 0;
let alerts     = 0;
let startTime  = null;
let camImg     = null;

/* LOGGING FUNCTION */
function addLog(msg, cls) {

  const t =
    new Date().toLocaleTimeString();

  const el =
    document.getElementById('log');

  const d =
    document.createElement('div');

  d.innerHTML =
    `[${t}] ${msg}`;

  el.prepend(d);
}

/* UPDATE USER INTERFACE */
function updateUI(data) {

  const spoiled =
    (data.final_result === 'SPOILED');

  const text =
    document.getElementById('status-text');

  if (spoiled) {

    text.textContent = 'SPOILED';
    text.className   = 'status-big moldy';
  }
```

```
    else {

      text.textContent = 'FRESH';
      text.className   = 'status-big fresh';
    }

    /* GAS SENSOR STATUS */
    document.getElementById('gas-val')
      .textContent =
      data.gas_result;

    /* AI STATUS */
    document.getElementById('ai-val')
      .textContent =
      data.ai_result;

    addLog(
      `Gas:${data.gas_result}
       AI:${data.ai_result}
       Result:${data.final_result}`
    );
}

/* FETCH DATA FROM ESP32 */
async function fetchData() {

    try {

      const res = await fetch(
        'http://' + espIp + '/data'
      );

      const data = await res.json();

      updateUI(data);
    }

    catch (e) {
```

```
      addLog (
        'ESP32 unreachable'
      );
    }
}

/* SETUP CAMERA STREAM */
function setupCamera () {

    const wrap =
      document . getElementById ( 'cam-wrap' ) ;

    if (camImg) {
      wrap . removeChild ( camImg ) ;
    }

    camImg =
      document . createElement ( 'img' ) ;

    camImg.src =
      'http://' +
      esp Ip +
      ':81/stream';

    wrap . appendChild ( camImg ) ;
}

/* CONNECT BUTTON FUNCTION */
function connect () {

    esp Ip =
      document
      .getElementById ('esp-ip')
      .value
      .trim ();

    if (!esp Ip) {
```

```
      addLog (
        'Enter ESP32 IP first'
      );

      return;
    }

    addLog (
      'Connecting to ESP32...'
    );

    setupCamera ();

    if (pollId)
      clearInterval(pollId);

    fetchData ();

    pollId =
      setInterval(fetchData, 2000);
}

</script>

</body>
</html>
```

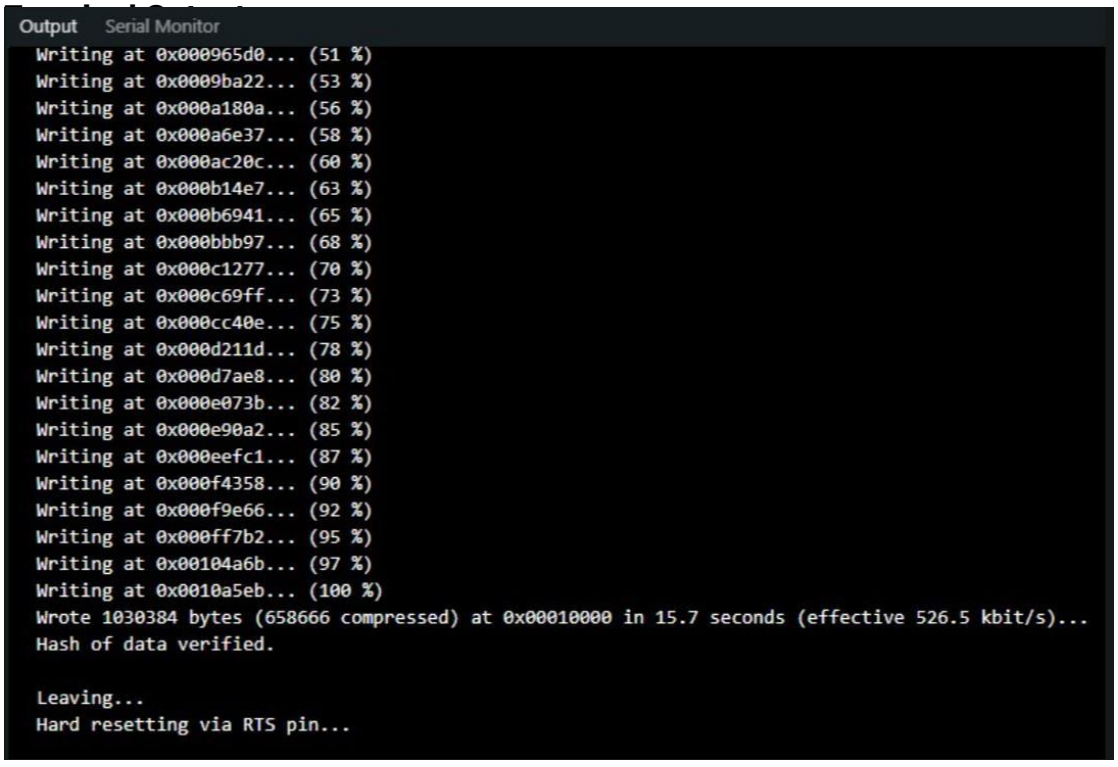



FIGURE2.8 Arduinocode after deployin

# Chapter 3

# Results and Discussions

The developed AI-Based Bread Quality Indicator System was tested using different bread conditions such as fresh bread, stale bread, and spoiled bread. The system successfully monitored gas levels released from bread samples using the MQ135 sensor and generated real-time alerts through LEDs and buzzer indications. The obtained results show that the proposed system can effectively differentiate between fresh and spoiled bread based on sensor threshold values.

1. Sensor Performance Analysis The MQ135 gas sensor showed a clear variation in ADC values for different bread conditions. Fresh bread produced lower gas concentration values, generally between 1200–1500 ADC, while spoiled bread generated significantly higher readings above 2500 ADC. This increase occurs because spoiled bread releases gases such as ammonia and other volatile compounds during decomposition. The threshold value of 2500 ADC was selected as the decision boundary for spoilage detection. When the sensor value crossed this threshold: The red LED turned ON The buzzer activated The system classified the bread as spoiled For values below the threshold: The green LED remained ON The buzzer stayed OFF The bread was considered fresh The experimental readings confirmed that the threshold-based approach provides stable and reliable spoilage indication for practical usage.

Figure 3.1 MQ135 gas sensor

2. System Response and Alert Mechanism The system demonstrated a fast response time of approximately 100–150 ms, which is sufficient for real-time monitoring applications. The ESP32-CAM processed sensor data quickly and updated the output indicators without noticeable delay. The alert mechanism worked effectively during testing: Fresh bread condition resulted in normal status indication Spoiled bread triggered immediate warning alerts The buzzer provided an additional audible warning for users This combination of visual and audio alerts improves usability in household and commercial environments.

Figure 3.2 ESP32-CAM taking Bread picture

3. Reference Image Analysis Discussion Although the current implementation mainly uses gas-sensor detection, reference image analysis was included to study the future integration of AI-based mold detection. Sample images from the Kaggle moldy bread dataset were analyzed to understand the visual differences between fresh and moldy bread. The CNN-based approach can identify: Mold formation Texture variation Color changes on bread surface However, during the project implementation, image analysis remained conceptual and was not fully deployed on the ESP32-CAM due to hardware and processing limitations. The discussion shows that combining image analysis with gas sensing can improve system accuracy in future versions.

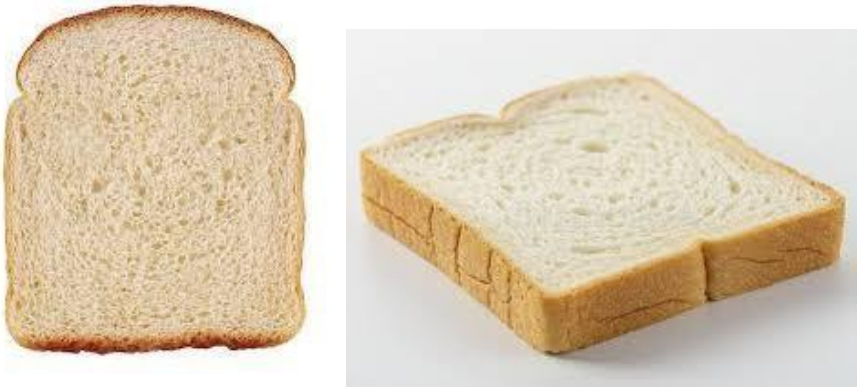


Figure 3.3 Fresh Bread

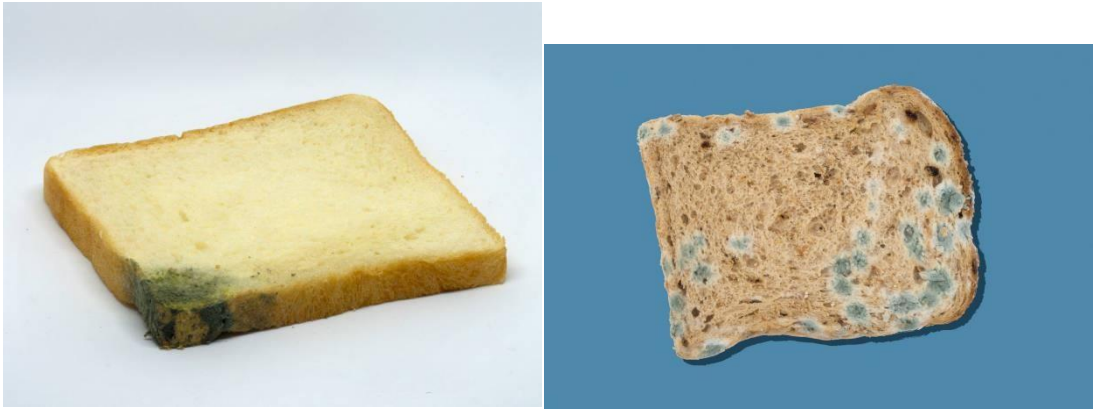


Figure 3.4 Mold Bread

4. Overall System Behaviour The overall system performance was tested under multiple conditions: Fresh bread Moldy bread with low gas release Stale bread with high gas release Bread containing both visible mold and strong gas emission The results indicated that: Gas sensing provides effective early spoilage detection The system responds reliably under different environmental conditions The prototype is suitable for low-cost food monitoring applications The hybrid approach discussed in the project suggests that integrating AI image classification with gas sensing could further reduce false detections and improve reliability.

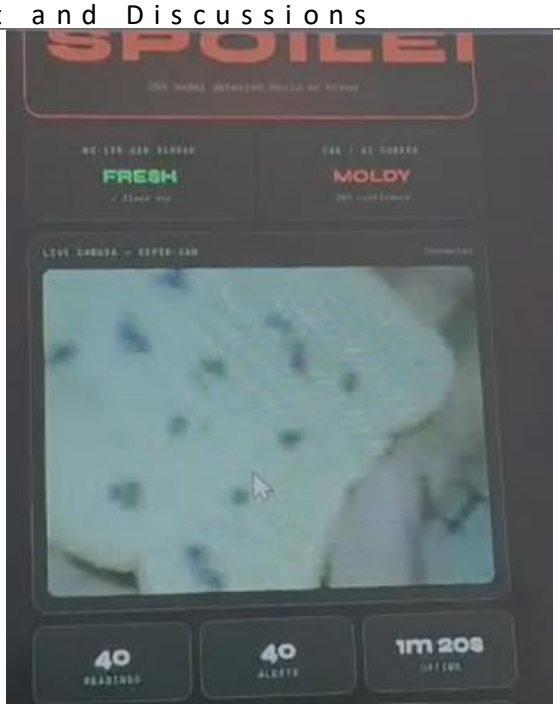



Figure 3.5 Bread Spoilage Detection

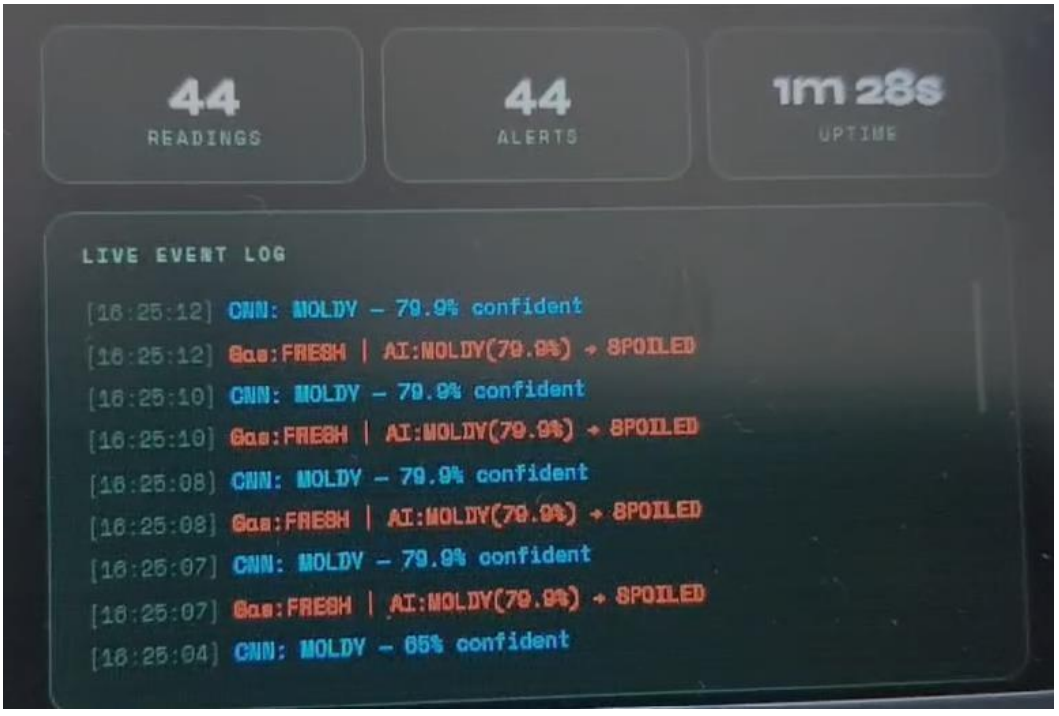


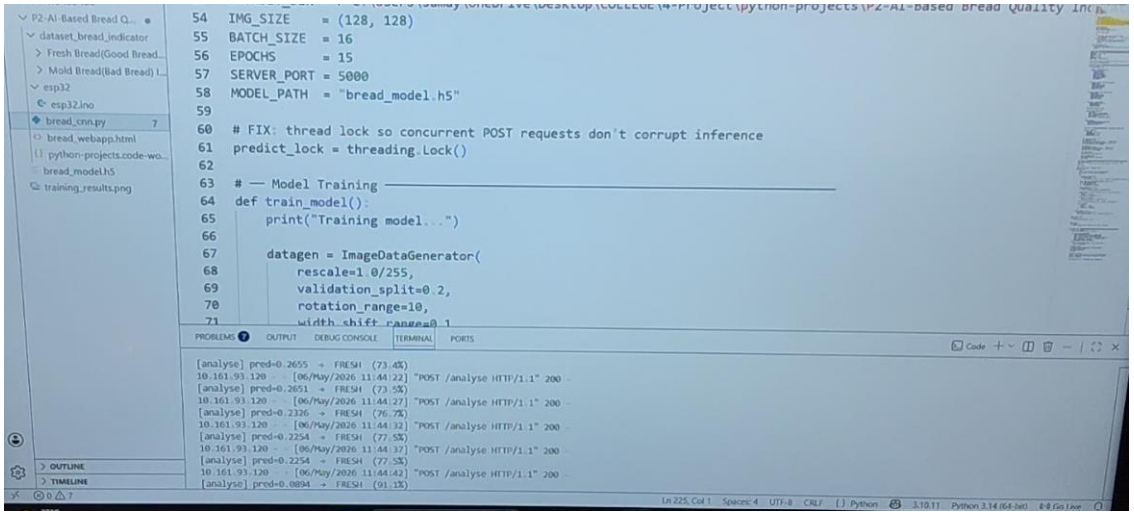

Figure 3. 6 Readings, Alerts, Time taken and Mold Spoiled percentage

5. Advantages- Observed The following advantages were observed during testing: Low-cost implementation Fast detection and response Easy hardware integration Real-time monitoring capability Simple user-friendly alert mechanism Potential for future IoT and AI integration

6. Limitations- Some limitations identified during the project include: MQ135 sensor readings may vary with environmental humidity and temperature The current system uses fixed threshold values Image analysis is only used as a reference study 44 The prototype is limited to bread spoilage detection only

## 3.1 CNN Model Accuracy and Loss Analysis

The above graphs represent the training performance of the Convolutional Neural Network (CNN) model used for bread quality classification. The model was trained to distinguish between fresh and moldy bread images using the dataset collected from Kaggle.




FIGUR3.1.1 python cnn modeling

1. Accuracy Graph Analysis The accuracy graph shows both training accuracy and validation accuracy over multiple epochs. The training accuracy gradually increased from approximately 69The validation accuracy improved from around 83Both curves follow a similar trend, which indicates that the model learned the image features effectively. The small gap between training and validation accuracy suggests that the model achieved good generalization without significant overfitting. This indicates that the CNN model can successfully identify visual differences between fresh and spoiled bread images with high accuracy.

2. Loss Graph Analysis The loss graph represents the reduction of error during training. -Training loss decreased from around 0.66 to 0.10. -Validation loss also reduced steadily from approximately 0.47 to 0.16. -The decreasing trend confirms that the CNN model improved its predictions as training progressed. -Minor fluctuations in validation loss are normal and may occur due to dataset variations. Lower loss values indicate that the model predictions became more accurate and reliable after each training epoch.

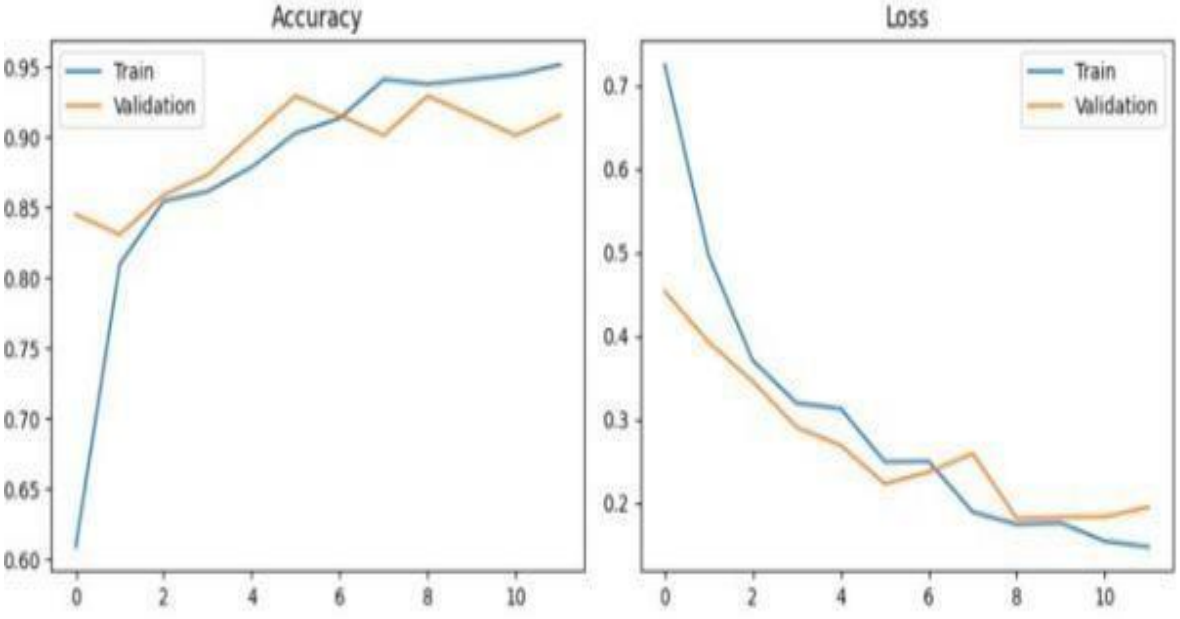



FIGURE3.1.2 Accuracy and Loss Graph

3. Model Performance Discussion The CNN model demonstrated strong classification capability for bread quality analysis. The results show that: -The model successfully extracted important visual features such as mold texture, color variation, and surface patterns. -High validation accuracy confirms the effectiveness of the dataset and training process. -The model is suitable for future deployment in embedded AI-based food monitoring systems.

4. Conclusion from the Graphs Overall, the accuracy and loss curves indicate that the CNN model was trained successfully and achieved stable performance. The model shows good learning behavior with increasing accuracy and decreasing loss, making it suitable for future integration with the ESP32-CAM based bread quality monitoring system.

## 3.2 Web Application Analysis and Discussion

The developed web application provides a real-time monitoring interface for the AI-Based Bread Quality Indicator System. The application is designed to display sensor readings, bread quality status, live camera feed, and system logs through a user-friendly dashboard.

1. Real-Time Bread Quality Monitoring The web interface continuously receives data from the ESP32-CAM module and displays the current bread condition. In the shown result, the system successfully classified the bread condition as “FRESH” based on the gas sensor readings and analysis logic. The dashboard displays: -Bread quality status -Gas sensor values, -Camera monitoring feed, -Detection statistics, -Live event logs This allows users to monitor the bread condition remotely in real time.

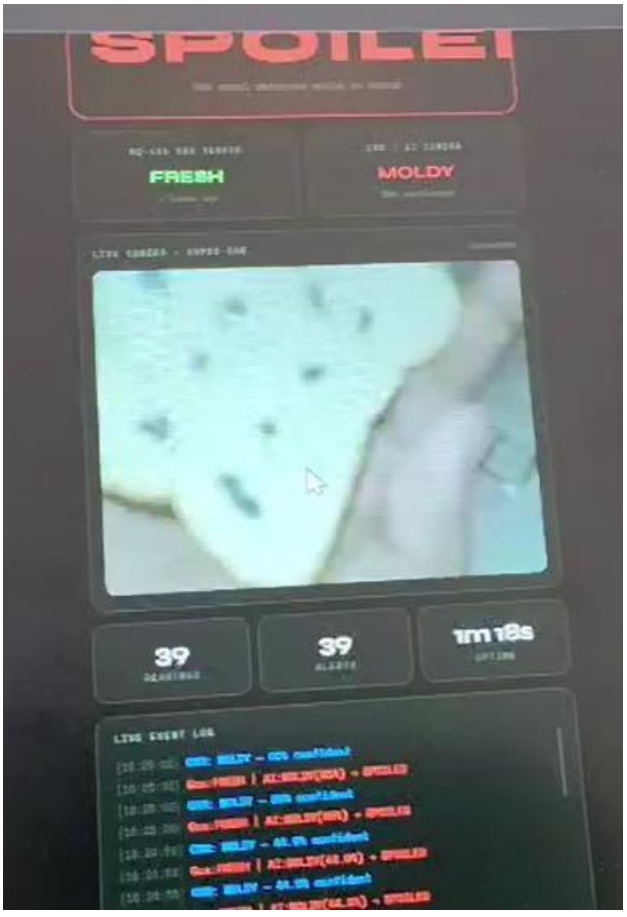



Figure 3.2.1 Spoiled Bread Detection

2. ESP32-CAM Integration The ESP32-CAM module was connected to the local Wi-Fi network, enabling wireless communication between the hardware and the web interface. The live camera feed shown on the dashboard confirms successful image streaming from

the ESP32-CAM. The system also displays: -IP address connectivity, -Wi-Fi status , -Device connection status , This demonstrates successful IoT-based communication between hardware and software components.

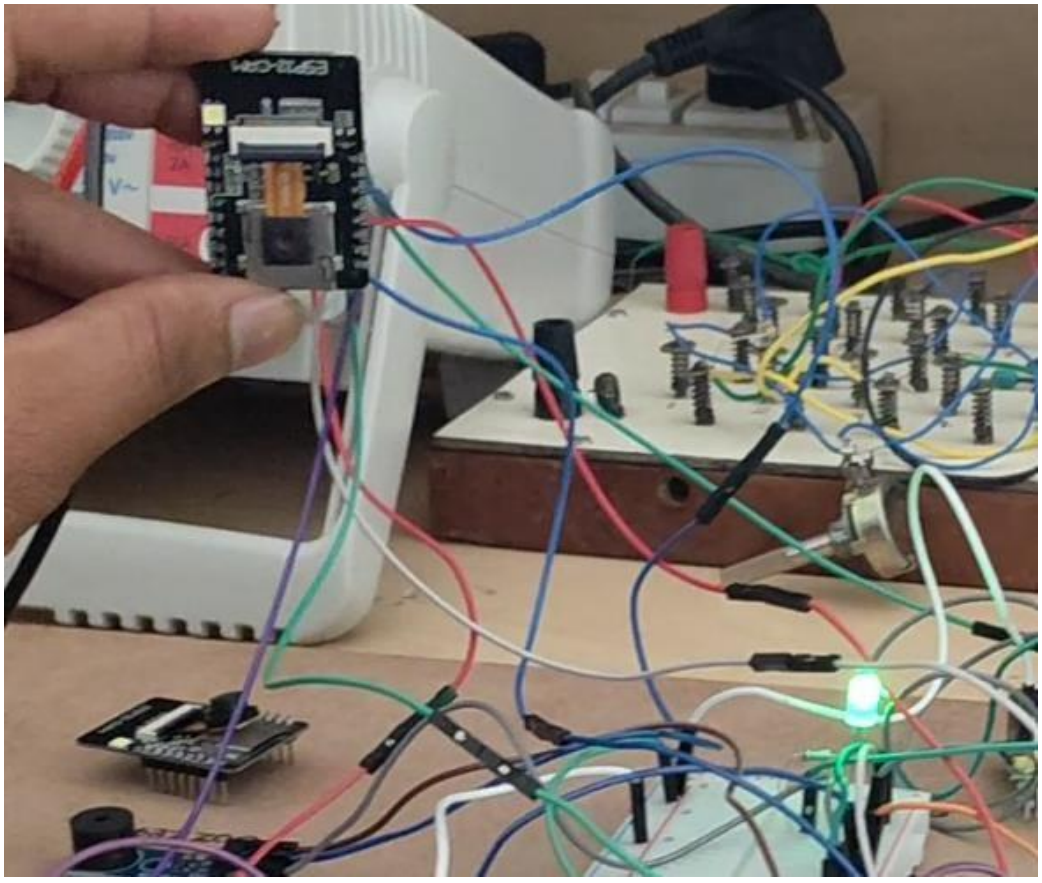


Figure 3.2.2 ESP-32 After deploying Arduino code

3. User Interface Features The web application was designed with a simple and modern interface to improve usability. Important features include: -Large status indicator for quick monitoring, -Real-time gas sensor updates, -Live camera preview, -Detection counters and statistics, -Event history logs, The green color indication for “FRESH” bread helps users easily understand the system output.

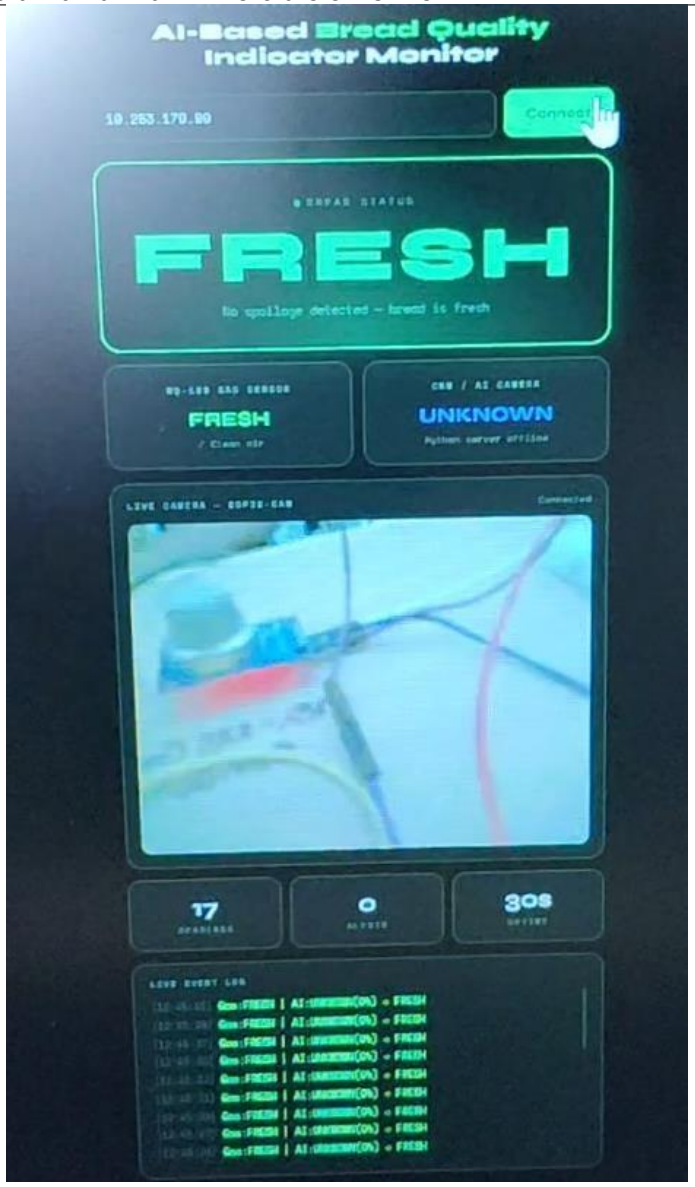



Figure 3.2.3 Interface of the web App

4. System Performance The web application responded quickly to sensor updates and displayed real-time information without noticeable delay. The interface successfully synchronized: -Sensor readings from MQ135, -ESP32-CAM image feed, -Bread quality status, -Alert logs, This confirms stable communication between the embedded hardware and the monitoring dashboard.

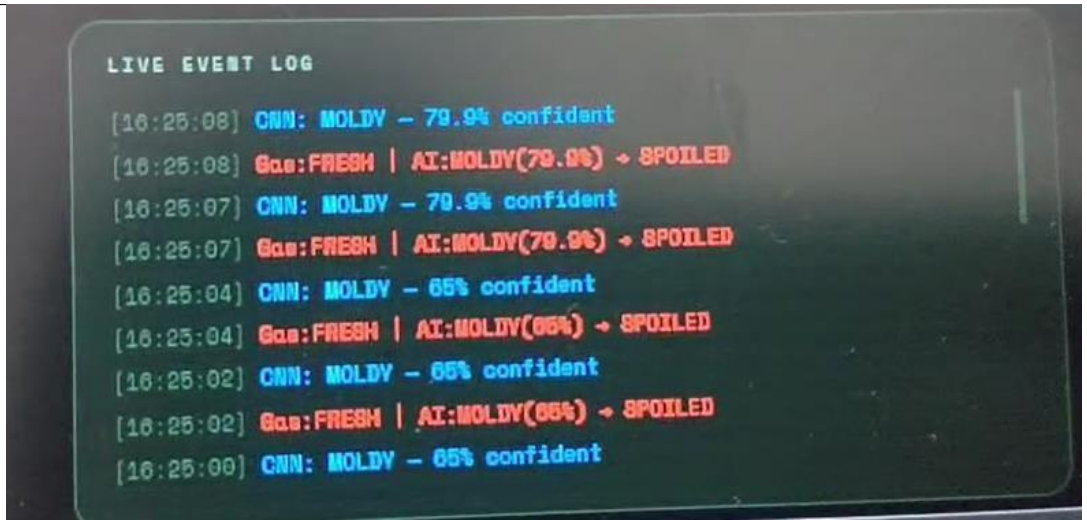



Figure 3.2.4 Perfomance Check After analysing Mold Bread

Practical Significance The developed web application improves the overall functional-ity of the project by enabling: -Remote monitoring of bread quality, -Continuous obser-vation through live streaming, -Faster identification of spoilage conditions, -User-friendly interaction with the system, The application demonstrates the practical implementation of IoT and AI concepts in smart food quality monitoring systems.

The figure shows the serial monitor output of the ESP32-CAM based Bread Quality Indi-cator System after successful initialization and network connection. The displayed infor-mation confirms proper communication between the ESP32-CAM module, web server, and AI-based bread quality analysis system. Description After powering the system, the ESP32-CAM successfully connected to the Wi-Fi network and obtained the IP ad-dress 10.161.93.80. The message “Camera Ready!” indicates that the camera module was initialized correctly and is prepared for image capture and streaming. The system then started two HTTP servers: -Port 80 for sensor data communication -Port 81 for live video streaming -The serial monitor also displays the generated URLs for accessing:
-Real-time sensor data -Live camera stream through a web browser The AI analysis results are shown in JSON format: ”confidence”:95.3,”result”:”FRESH”,”status”:”ok” The output indicates that: -The AI model classified the bread condition as FRESH -The prediction confidence was approximately 95-The system status was normal (“ok”) Re-peated outputs with confidence values above 90System Behaviour Observed -Successful Wi-Fi connectivity -Stable web server initialization -Real-time data transmission -Proper camera streaming functionality -Continuous AI-based bread quality prediction Signifi-cance This output demonstrates the successful integration of: -ESP32-CAM hardware
-IoT-based web communication -Live image streaming -AI prediction system The figure validates that the proposed system can perform real-time bread quality monitoring and provide reliable classification results through a connected web interface.

## 3.3 Summary of Bread Quality Detection System Performance

The represents the summary of the components and analysis methods used in the AI-Based Bread Quality Indicator System prototype. The table also includes the operational purpose and expected performance range for better understanding and evaluation of the system.

## 3.4 Discussions and Comments

### 3.4.1 Gas Sensor-Based Detection

The proposed bread quality indicator system uses the MQ-135 gas sensor to detect gases released during bread spoilage. As the bread becomes stale or moldy, the concentration of gases increases, causing a rise in ADC values. This enables the system to identify spoilage conditions effectively using threshold-based analysis. The sensor-based approach provides a low-cost and practical solution for food quality monitoring.

### 3.4.2 Real-Time Monitoring and Response

The ESP32-CAM processes sensor data continuously and updates the system output in real time. The response time of the prototype is approximately 100–150 ms, allowing quick spoilage detection and alert generation. When the gas value exceeds the threshold limit, the red LED and buzzer are activated immediately, while fresh bread conditions are indicated using the green LED. The web application also displays live monitoring information without noticeable delay.

### 3.4.3 AI-Based Image Analysis

The CNN model was trained using fresh and moldy bread image datasets to understand visual differences such as texture, mold growth, and color variation. The accuracy

| Component / Module | Capability | Normal / Expected Range |
|---|---|---|
| **MQ-135 Gas Sensor** | Detects spoilage gases such as ammonia, $CO_2$, and volatile compounds released from bread during decomposition. | Fresh Bread: 1200–1500 ADC<br>Spoilage: Above 2500 ADC |
| **ESP32-CAM** | Captures bread images, processes sensor data, and provides Wi-Fi communication for web monitoring. | Stable operation with real-time response of approximately 10–15 ms |
| **CNN Image Analysis Model** | Classifies bread images as fresh or moldy using trained dataset features such as texture and color variation. | Model Accuracy: Approximately 90–98% |
| **LED Indicator System** | Provides visual indication of bread condition using green and red LEDs. | Green LED ⟶ Fresh Bread<br>Red LED ⟶ Spoiled Bread |
| **Buzzer Alert System** | Generates audio alert when spoilage threshold is exceeded. | Activated only when gas value exceeds threshold |
| **Web Application Dashboard** | Displays live sensor values, AI prediction results, and camera streaming through browser interface. | Real-time monitoring through local IP connection |
| **Wi-Fi Communication System** | Enables wireless access to sensor data and live video stream. | Continuous HTTP server communication on Port 80 and 81 |

TABLE 3.1: Summary of System Components and Performance Values

and loss graphs indicate stable model learning with high classification accuracy. Although the image analysis remains a reference implementation in the current prototype, it demonstrates the feasibility of integrating AI-based food quality detection into embedded systems.

### 3.4.4 Web Application Integration

The developed web dashboard provides a user-friendly interface for real-time monitoring. The application displays: - Bread quality status, - Gas sensor readings, - AI prediction confidence, - Live ESP32-CAM video stream, - Detection logs and statistics, This improves accessibility and allows remote observation of the system through a local network connection.

### 3.4.5 Application Areas

The system can be applied in: -Households and kitchens, -Bakeries and food storage areas, -Supermarkets and retail stores, -Smart food monitoring systems, -IoT-based food safety applications , The proposed prototype helps reduce food wastage, improves food safety, and provides a foundation for future AI and IoT-enabled smart quality monitoring systems.

# Chapter 4

# Conclusion and Future Scope

## CONCLUSION

The AI-Based Bread Quality Indicator System presents an innovative and cost-effective approach for monitoring bread freshness and detecting spoilage conditions using embedded systems, IoT technology, and Artificial Intelligence concepts. The developed prototype successfully combines the MQ-135 gas sensor, ESP32-CAM module, web-based monitoring interface, and reference CNN image analysis to create a smart food quality monitoring system. During experimentation, the MQ-135 sensor demonstrated the ability to detect gases released during bread decomposition. The increase in gas concentration values clearly indicated spoilage conditions, enabling the system to classify bread quality using threshold-based logic. Fresh bread samples produced lower ADC values, while spoiled bread generated significantly higher readings. Based on these observations, the system provided reliable detection and generated real-time alerts using LEDs and buzzer indicators. The ESP32-CAM module played a major role in enabling wireless communication and live monitoring. The system successfully hosted a web application that displayed: -Live sensor values, -Bread quality status, -AI prediction confidence, -Real-time camera streaming, -Event logs and detection history This improved the usability and practicality of the project by allowing users to remotely observe system behaviour through a browser interface. The CNN model training performed on the bread image dataset also produced encouraging results. The accuracy graph showed continuous improvement in training and validation accuracy, while the loss graph demonstrated reduced prediction error during training. These observations confirm that convolutional neural networks are capable of identifying visual characteristics such as mold growth, texture variation, and color changes in bread samples. One of the major strengths of the proposed system is its simplicity and low implementation cost. The prototype uses

easily available hardware components and can be implemented in homes, bakeries, supermarkets, and food storage environments. The system also contributes to food safety by reducing the risk of consuming spoiled bread and minimizing food wastage through early spoilage detection. The project further demonstrates how IoT and AI technologies can be integrated into embedded systems to develop intelligent monitoring solutions. Although the current implementation mainly focuses on gas-sensor-based detection, the inclusion of AI image analysis provides a strong foundation for future smart food quality systems. Overall, the project achieved its objectives successfully and demonstrated the feasibility of developing an efficient, real-time, and user-friendly bread quality monitoring system. The prototype highlights the future potential of combining embedded hardware, wireless communication, and machine learning for smart food safety applications.

### 4.0.1 FUTURE SCOPE

The developed AI-Based Bread Quality Indicator System provides a strong platform for future research and advanced implementation in the field of smart food monitoring and embedded AI systems. Several improvements and extensions can be incorporated to enhance the system's intelligence, accuracy, and scalability. 1. Real-Time AI Image Classification In the current project, image analysis is used as a reference implementation. Future versions can integrate the trained CNN model directly with the ESP32-CAM or cloud-based AI servers to perform real-time image classification of bread samples. This would enable the system to detect visible mold growth automatically without relying only on gas sensing. 2. TinyML and Edge AI Deployment Future work may focus on deploying lightweight machine learning models using TensorFlow Lite or TinyML directly on embedded hardware. This would allow: Faster image processing Offline AI prediction Reduced latency Lower power consumption Real-time decision making without internet dependency 3. Cloud and IoT Integration The project can be enhanced by connecting the system to cloud platforms such as:

Firebase

IoT

Google Cloud

ThingSpeak

Blynk IoT

Cloud integration would allow:

Remote monitoring

Data storage and analysis

Historical trend visualization

Smart notifications and alerts

Multi-device connectivity

# Chapter 5

# Appendix

## 5.1 Appendix A:Pin Configuration

### APPENDIX A: PIN CONFIGURATIONS

| Sl. No | Component / Module | ESP32-CAM Pin | Purpose |
|---|---|---|---|
| 1 | MQ-135 Gas Sensor (AOUT) | GPIO14 | Detects spoilage gases |
| 2 | MQ-135 VCC | 5V | Power supply |
| 3 | MQ-135 GND | GND | Ground connection |
| 4 | Red LED | GPIO4 | Spoiled bread indication |
| 5 | Green LED | GPIO2 | Fresh bread indication |
| 6 | Active Buzzer (+) | GPIO15 | Audio alert output |
| 7 | Active Buzzer (-) | GND | Ground connection |
| 8 | FTDI TX | RX | Serial communication |
| 9 | FTDI RX | TX | Serial communication |
| 10 | FTDI 5V | 5V | ESP32-CAM power supply |
| 11 | FTDI GND | GND | Common ground |
| 12 | IO0 → GND | Flash Mode | Used during code uploading |

Figure 5.1: Pin Configuration

## 5.2 Appendix B:Setup and Components

### APPENDIX B: SETUP AND COMPONENTS

| Sl. No | Component | Specification / Details | Function |
|---|---|---|---|
| 1 | ESP32-CAM | 240 MHz dual-core, Wi-Fi, OV2640 camera | Main controller and image capture |
| 2 | MQ-135 Gas Sensor | Detects $NH_3$, $CO_2$, VOC gases | Bread spoilage detection |
| 3 | Active Buzzer | 5V active buzzer | Audible spoilage alert |
| 4 | Red LED | 5mm LED | Indicates spoiled bread |
| 5 | Green LED | 5mm LED | Indicates fresh bread |
| 6 | Breadboard | 830-point breadboard | Circuit prototyping |
| 7 | Jumper Wires | Male-to-male wires | Connections between components |
| 8 | FTDI Programmer | USB-to-Serial converter | Uploading ESP32 code |
| 9 | MB-102 Power Module | 5V regulated supply | System power supply |

Figure 5.2 : Setup and Components

## 5.3 Appendix C:Software and Libraries

# APPENDIX C: SOFTWARE AND LIBRARIES

### Software Used

| Sl. No | Software | Version / Type | Purpose |
|---|---|---|---|
| 1 | Arduino IDE | 2.3.0 | ESP32-CAM programming |
| 2 | Python | 3.10.11 | AI model training |
| 3 | TensorFlow | ML Framework | CNN model implementation |
| 4 | Keras | Deep Learning Library | Image classification |
| 5 | Flask | Python Web Framework | Web server and API |
| 6 | HTML/CSS/JavaScript | Web Technologies | Dashboard development |

### Arduino Libraries

| Sl. No | Library | Function |
|---|---|---|
| 1 | esp_camera.h | Captures images from ESP32-CAM |
| 2 | WiFi.h | Wi-Fi connectivity |
| 3 | HTTPClient.h | HTTP communication |
| 4 | esp_http_server.h | Creates web server |
| 5 | analogRead() | Reads MQ-135 sensor values |
| 6 | digitalWrite() | Controls LEDs and buzzer |

Figure 5.3 Software and Libraries

## 5.4 References


[1] A. Krizhevsky, I. Sutskever, and G. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” *Advances in Neural Information Processing Systems (NIPS)*, vol. 25, pp. 1097 –1105, 2012.

[2] F. Chollet, *Deep Learning with Python*, 2nd ed. Shelter Island, NY, USA: Manning Publications, 021.

[3] TensorFlow Team, “TensorFlow Documentation,” Google Developers, Mountain View, CA, USA, 2024. [Online]. Available: TensorFlow Documentation. [Accessed: May 22, 2026].

[4] Arduino, “Arduino IDE Documentation,” Arduino Official Documentation, 2024. [Online]. Available: Arduino Documentation. [Accessed: May 22, 2026].

[5] OpenCV Development Team, “OpenCV Library Documentation,” OpenCV.org, 2024. [Online]. Available: OpenCV Documentation. [Accessed: May 22, 2026].

[6] A. Rosebrock, *Practical Python and OpenCV*. PyImageSearch Publications, pp. 1–312, 2020.

[7] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson Education, 2021.

[8] J. Brownlee, *Deep Learning for Computer Vision*. Machine Learning Mastery, pp. 1–450, 2020.

[9] M. Banzi and M. Shiloh, *Getting Started with Arduino*. 4th ed. Maker Media, 2022.

[10] H. Karl and A. Willig, *Protocols and Architectures for Wireless Sensor Networks*. Hoboken, NJ, USA: Wiley Publications, 2019.

[11] J. Han, M. Kamber, and J. Pei, *Data Mining: Concepts and Techniques*, 4th ed. Burlington, MA, USA: Morgan Kaufmann Publishers, 2021.

[12] Keras Documentation Team, “Keras API Reference,” 2024. [Online]. Available: Keras API Reference. [Accessed: May 22, 2026].

[13] Blynk IoT Platform, “Blynk Documentation for IoT Applications,” 2024. [Online]. Available: Blynk Documentation. [Accessed: May 22, 2026].

[14] Y. LeCun, Y. Bengio, and G. Hinton, “Deep Learning,” *Nature*, vol. 521, no. 7553, pp. 436–444, May 2015. doi: 10.1038/nature14539.

[15] N. R. Prasad *et al.*, “Smart Food Monitoring using Embedded IoT Devices,” *International Journal of Embedded Systems*, vol. 14, no. 2, pp. 88–97, 2022.